

# ArduPilot PNT (Positioning, Navigation, Timing) Reference Audit

**Scope.** This document is an audit-grade, read-only static-analysis catalog of every location in the ArduPilot firmware codebase (repository root containing ArduCopter/, ArduPlane/, Rover/, ArduSub/, AntennaTracker/, Blimp/, libraries/, modules/) where Positioning, Navigation, and Timing (PNT) data is handled, together with a two-layer dependency map for every catalogued reference. GROUP 1 catalogs core references — code that directly implements, reads, writes, or processes PNT data. GROUP 2 catalogs indirect / relational references — code that does not directly read or write PNT data but whose behavior changes as a result of PNT state. All file paths are repository-root-relative; every code snippet is reproduced VERBATIM from the cited path and line range so each row is independently verifiable. No source file is modified by this audit.

## Legend

**Role (Group 1)** Source = produces/writes PNT state from a measurement | Transform = fuses/derives/converts PNT state (EKF fusion, GPS blending, geodetic↔NED) | Sink = only reads PNT state to drive control/behavior.

**Group 2 columns** Trigger Condition (what invokes the code) | PNT State Observed (the exact accessor/value read, e.g. `gps.status()`, `ahrs.have_inertial_nav()`, EKF variance) — the discriminator from Group 1 | Behavior Change Description (how behavior changes).

**Dependency Type (Layer 1)** Data dependency (shared data value/struct field) | Function call (direct call) | Inheritance / Interface (backend overrides a virtual frontend interface) | Shared global / singleton (access via an AP:: accessor — `AP::gps()`, `AP::ahrs()`, `AP::scheduler()`, `AP::rtc()`).

**Notes flags** [CIRCULAR] circular dependency (e.g. `AHRS↔EKF`) | [SHARED-STRUCT] PNT fields packed with non-PNT fields (extraction risk) | [DUPLICATED] copy-pasted PNT logic across locations | [MULTI-CATEGORY] spans >1 PNT category (emitted once per table) | [ABSENT] expected PNT element not found where checked.

**Cross-reference** Each main table owns a monotonic, per-table # space (1,2,3...). Its dependency sub-table (the matching Xa table) reuses those exact numbers as Ref #. Each Xa table contains a Layer 1 block (direct edges) followed by a Layer 2 block (full transitive chain + final consumer + Chain Depth = the number of → hops explicitly listed in that chain).

**Footprint surveyed:** libraries/ = 151 subsystems; libraries/AP\_GPS/\*.{cpp,h} = 44 files (14 receiver backends each catalogued individually + AP\_GPS\_Blended + GPS\_Backend base); EKF families

AP\_NavEKF/AP\_NavEKF2/AP\_NavEKF3 with AP\_NavEKF3 split across 13 fusion modules (all catalogued); flight/drive-mode files = 81 (ArduCopter 29 + ArduPlane 26 + Rover 14 + ArduSub 12), enumerated in the Coverage & Explicit-Absence Register. PNT state is produced under libraries/ and consumed/acted-upon in vehicle glue code (failsafe, ekf\_check, fence, flight-mode gating).

## GROUP 1 — CORE PNT REFERENCES

Code that directly implements, reads, writes, or processes PNT data. Role is one of Source / Transform / Sink.

**Table 1 — Core Positioning (GPS receiver backends, Location, inertial position, position controllers, non-GPS sources)**

| # | File Path                         | Line(s)   | Function / Class                          | Role   | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                              | Notes                                                                                                                                                                                       |
|---|-----------------------------------|-----------|-------------------------------------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | libraries/AP_GPS/AP_GPS_UBLOX.cpp | 1510-1518 | AP_GPS_UBLOX::parse_gps()<br>(NAV-POSLLH) | Source | <pre>         _last_pos_time = _buffer.posllh.itow;         state.location.lng = _buffer.posllh.longitude;         state.location.lat = _buffer.posllh.latitude;         state.have_undulation = true;         state.undulation = (_buffer.posllh.altitude_msl - _buffer. posllh.altitude_ellipsoid) * 0.001;         set_alt_amsl_cm(state, _buffer.posllh.altitude_msl / 10);          state.status = next_fix;         _new_position = true; </pre> | Receiver backend writes GPS_State.location/status from the u-blox UBX binary NAV-POSLLH. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.       |
| 2 | libraries/AP_GPS/AP_GPS_NMEA.cpp  | 333-340   | AP_GPS_NMEA::_term_complete()             | Source | <pre>         state.location.lat = _new_latitude;         state.location.lng = _new_longitude;     }     state.num_sats = _new_satellite_count;     state.hdop = _new_hdop;     switch(_new_quality_indicator) {     case 0: // Fix not available or invalid         state.status = AP_GPS::NO_FIX; </pre>                                                                                                                                             | Receiver backend writes GPS_State.location/status from the NMEA-0183 ASCII (GGA/RMC) sentence. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State. |
| 3 | libraries/AP_GPS/AP_GPS_SBF.cpp   | 484-491   | AP_GPS_SBF::process_message()             | Source | <pre>         state.location.lat = (int32_t)(temp.Latitude * RAD_TO_ DEG_DOUBLE * (double)1e7);         state.location.lng = (int32_t)(temp.Longitude * RAD_TO _DEG_DOUBLE * (double)1e7);         state.have_undulation = !is_DNU(temp.Undulation);         double height = temp.Height; // in metres         if (state.have_undulation) {             height -= temp.Undulation;             state.undulation = -temp.Undulation;         } </pre>   | Receiver backend writes GPS_State.location/status from the Septentrio SBF binary PVTGeodetic. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.  |
| 4 | libraries/AP_GPS/AP_GPS_NOVA.cpp  | 209-216   | AP_GPS_NOVA::process_message()            | Source | <pre>         state.last_gps_time_ms = AP_HAL::millis();          state.location.lat = (int32_t) (bestposu.lat * (double)1e7 );         state.location.lng = (int32_t) (bestposu.lng * (double)1e7 );          state.have_undulation = true;         state.undulation = -bestposu.undulation;         set_alt_amsl_cm(state, bestposu.hgt * 100); </pre>                                                                                               | Receiver backend writes GPS_State.location/status from the NovAtel OEM BESTPOS. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.                |
| 5 | libraries/AP_GPS/AP_GPS_SBP.cpp   | 276-283   | AP_GPS_SBP::_attempt_state_update()       | Source | <pre>         state.location.lat = (int32_t) (pos_llh-&gt;lat * (doubl e)1e7);         state.location.lng = (int32_t) (pos_llh-&gt;lon * (doubl e)1e7);         state.location.alt = (int32_t) (pos_llh-&gt;height * 100 );         state.num_sats = pos_llh-&gt;n_sats;          if (pos_llh-&gt;flags == 0) {             state.status = AP_GPS::GPS_OK_FIX_3D; </pre>                                                                               | Receiver backend writes GPS_State.location/status from the Swift Navigation SBP v1 POS_LLH. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.    |

| #  | File Path                        | Line(s) | Function / Class                    | Role   | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                           | Notes                                                                                                                                                                                    |
|----|----------------------------------|---------|-------------------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6  | libraries/AP_GPS/AP_GPS_SBP2.cpp | 328-335 | AP_GPS_SBP2::_attmpt_state_update() | Source | <pre> // state.location.lat = (int32_t) (last_pos_llh.lat * (double)1e7); state.location.lng = (int32_t) (last_pos_llh.lon * (double)1e7); state.location.alt = (int32_t) (last_pos_llh.height * 100); state.num_sats = last_pos_llh.n_sats;  switch (last_pos_llh.flags.fix_mode) { case 1: </pre>                                                                                                                                                 | Receiver backend writes GPS_State.location/status from the Swift Navigation SBP v2 POS_LLH. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State. |
| 7  | libraries/AP_GPS/AP_GPS_SIRF.cpp | 178-185 | AP_GPS_SIRF::_parse_gps()           | Source | <pre> state.status = AP_GPS::NO_FIX; }else if ((_buffer.nav.fix_type &amp; FIX_MASK) == FIX_3D) { state.status = AP_GPS::GPS_OK_FIX_3D; }else{ state.status = AP_GPS::GPS_OK_FIX_2D; } state.location.lat = int32_t(be32toh(_buffer.nav.latitude)); state.location.lng = int32_t(be32toh(_buffer.nav.longitude)); </pre>                                                                                                                            | Receiver backend writes GPS_State.location/status from the SiRF binary geodetic navigation. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State. |
| 8  | libraries/AP_GPS/AP_GPS_ERB.cpp  | 150-157 | AP_GPS_ERB::_parse_gps()            | Source | <pre> _last_pos_time = _buffer.pos.time; state.location.lng = (int32_t)(_buffer.pos.longitude * (double)1e7); state.location.lat = (int32_t)(_buffer.pos.latitude * (double)1e7); state.have_undulation = true; state.undulation = _buffer.pos.altitude_msl - _buffer.pos.altitude_ellipsoid; set_alt_amsl_cm(state, _buffer.pos.altitude_msl * 100); state.status = next_fix; _new_position = true; </pre>                                         | Receiver backend writes GPS_State.location/status from the Emlid Reach ERB protocol. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.        |
| 9  | libraries/AP_GPS/AP_GPS_MAV.cpp  | 67-75   | AP_GPS_MAV::_handle_msg()           | Source | <pre> state.status = (AP_GPS::GPS_Status)packet.fix_type;  Location loc = {}; loc.lat = packet.lat; loc.lng = packet.lon; if (have_alt) { loc.alt = packet.alt * 100; // convert to centimeters } state.location = loc; </pre>                                                                                                                                                                                                                      | Receiver backend writes GPS_State.location/status from the MAVLink GPS_INPUT message. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.       |
| 10 | libraries/AP_GPS/AP_GPS_MSP.cpp  | 44-51   | AP_GPS_MSP::_handle_msp()           | Source | <pre> state.status = (AP_GPS::GPS_Status)pkt.fix_type; state.num_sats = pkt.satellites_in_view; state.location = { pkt.latitude, pkt.longitude, pkt.msl_altitude, Location::AltFrame::ABSOLUTE }; </pre>                                                                                                                                                                                                                                            | Receiver backend writes GPS_State.location/status from the MSP protocol GPS data. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.           |
| 11 | libraries/AP_GPS/AP_GPS_GSOF.cpp | 231-238 | AP_GPS_GSOF::_pack_state_data()     | Source | <pre> state.location.lat = (int32_t)(RAD_TO_DEG_DOUBLE * position.latitude_rad * (double)1e7); state.location.lng = (int32_t)(RAD_TO_DEG_DOUBLE * position.longitude_rad * (double)1e7); state.location.alt = (int32_t)(position.altitude * 100); state.last_gps_time_ms = AP_HAL::millis();  if ((vel.velocity_flags &amp; 1) == 1) { state.ground_speed = vel.horizontal_velocity; state.ground_course = wrap_360(degrees(vel.heading)); } </pre> | Receiver backend writes GPS_State.location/status from the Trimble GSOF binary. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.             |

| #  | File Path                                 | Line(s) | Function / Class                        | Role      | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Notes                                                                                                                                                                                                                              |
|----|-------------------------------------------|---------|-----------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12 | libraries/AP_GPS/AP_GPS_DroneCAN.cpp      | 388-396 | AP_GPS_DroneCAN::handle_fix2_msg()      | Source    | <pre>         if (msg.height_msl_mm != msg.height_ellipsoid_mm) {             seen_valid_height_ellipsoid = true;         }         interim_state.have_undulation = seen_valid_height_ellipsoid;          if (seen_valid_height_ellipsoid) {             interim_state.undulation = (msg.height_msl_mm - msg.height_ellipsoid_mm) * 0.001;         }         interim_state.location = loc;         set_alt_amsl_cm(interim_state, alt_amsl_cm); </pre>                                               | Receiver backend writes GPS_State.location/status from the DroneCAN/UAVCAN Fix2 (external boundary modules/DroneCAN). One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.                 |
| 13 | libraries/AP_GPS/AP_GPS_ExternalAHRSS.cpp | 49-56   | AP_GPS_ExternalAHRSS::handle_external() | Source    | <pre> state.num_sats = pkt.satellites_in_view;  const Location loc {     pkt.latitude,     pkt.longitude,     pkt.msl_altitude,     Location::AltFrame::ABSOLUTE }; </pre>                                                                                                                                                                                                                                                                                                                           | Receiver backend writes GPS_State.location/status from the External AHRSS/INS GPS feed. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.                                               |
| 14 | libraries/AP_GPS/AP_GPS_SITL.cpp          | 86-93   | AP_GPS_SITL::read()                     | Source    | <pre> state.status = AP_GPS::GPS_OK_FIX_3D; state.num_sats = 15;  state.location = Location{     int32_t(latitude*1e7),     int32_t(longitude*1e7),     int32_t(altitude*100),     Location::AltFrame::ABSOLUTE }; </pre>                                                                                                                                                                                                                                                                            | Receiver backend writes GPS_State.location/status from the Software-in-the-loop simulated GPS. One of 14 receiver backends, each catalogued individually. [SHARED-STRUCT] target GPS_State.                                        |
| 15 | libraries/AP_GPS/GPS_Backend.cpp          | 101-109 | AP_GPS_Backend::fill_3d_velocity()      | Transform | <pre> void AP_GPS_Backend::fill_3d_velocity(void) {     float gps_heading = radians(state.ground_course);      state.velocity.x = state.ground_speed * cosf(gps_heading);     state.velocity.y = state.ground_speed * sinf(gps_heading);     state.velocity.z = 0;     state.have_vertical_velocity = false; } </pre>                                                                                                                                                                                | Backend base class; derives state.velocity (NED) from ground_speed + ground_course for receivers that do not report 3D velocity. Shared by all backends.                                                                           |
| 16 | libraries/AP_GPS/AP_GPS.h                 | 324-329 | AP_GPS::location()                      | Source    | <pre> const Location &amp;location(uint8_t instance) const {     return state[instance].location; } const Location &amp;location() const {     return location(primary_instance); } </pre>                                                                                                                                                                                                                                                                                                           | Frontend accessor exposing backend-measured GPS_State.location; dispatches to primary_instance. Reached vehicle-side via the AP::gps() singleton.                                                                                  |
| 17 | libraries/AP_GPS/AP_GPS.h                 | 360-365 | AP_GPS::velocity()                      | Source    | <pre> const Vector3f &amp;velocity(uint8_t instance) const {     return state[instance].velocity; } const Vector3f &amp;velocity() const {     return velocity(primary_instance); } </pre>                                                                                                                                                                                                                                                                                                           | 3D NED velocity accessor of measured GPS_State.velocity. [MULTI-CATEGORY] velocity also drives Navigation fusion (Table 2).                                                                                                        |
| 18 | libraries/AP_GPS/AP_GPS.h                 | 192-201 | struct AP_GPS::GPS_State                | Source    | <pre> struct GPS_State {     uint8_t instance; // the instance number of this GPS      // all the following fields must all be filled by the backend driver     GPS_Status status; // driver fix status     uint32_t time_week_ms; // GPS time (millisecons from start of GPS week)     uint16_t time_week; // GPS week number     Location location; // last fix location     float ground_speed; // ground speed in m/s     float ground_course; // ground course in degrees, wrapped 0-360 </pre> | [SHARED-STRUCT][MULTI-CATEGORY] One struct mixes Positioning (location), Navigation (velocity, ground_speed, ground_course) and Timing (time_week_ms, time_week) — see Table 3 #8. Primary extraction-risk struct for a PNT split. |

| #  | File Path                                      | Line(s) | Function / Class                          | Role      | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Notes                                                                                                                                                                                                          |
|----|------------------------------------------------|---------|-------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 19 | libraries/AP_GPS/AP_GPS_Blended.cpp            | 17-25   | AP_GPS_Blended::_calc_weights()           | Transform | <pre> bool AP_GPS_Blended::_calc_weights(void) {     // zero the blend weights     memset(&amp;_amp;blend_weights, 0, sizeof(_amp;blend_weights));      static_assert(GPS_MAX_RECEIVERS == 2, "GPS blending only currently works with 2 receivers");     // Note that the early quit below relies upon exactly 2 instances     // The time delta calculations below also rely upon every instance being currently detected and being parsed </pre>                                                                                                                                                                                                                                              | Multi-receiver blending of location/velocity (GPS_MAX_RECEIVERS==2). Reads gps.state[].status (Data dependency) to weight receivers.                                                                           |
| 20 | libraries/AP_Common/Location.h                 | 11-20   | class Location                            | Source    | <pre> uint8_t relative_alt : 1; // 1 if altitude is relative to home uint8_t loiter_ccw : 1; // 0 if clockwise, 1 if counter clockwise uint8_t terrain_alt : 1; // this altitude is above terrain uint8_t origin_alt : 1; // this altitude is above ekf origin uint8_t loiter_xtrack : 1; // 0 to crosstrack from center of waypoint, 1 to crosstrack from tangent exit location  // note that mission storage only stores 24 bits of altitude (~ +/- 83km) int32_t alt; // in cm int32_t lat; // in 1E7 degrees int32_t lng; // in 1E7 degrees </pre>                                                                                                                                          | [SHARED-STRUCT] Geodetic PNT fields lat/lng (1E7 deg) and alt (cm) packed with NON-PNT bitfields relative_alt, loiter_ccw, terrain_alt, origin_alt, loiter_xtrack. Ubiquitous value type; top extraction risk. |
| 21 | libraries/AP_Common/Location.cpp               | 259-268 | Location::get_vector_from_origin_NEU_cm() | Transform | <pre> bool Location::get_vector_from_origin_NEU_cm(T &amp;vec_neu) const {     // convert altitude     int32_t alt_above_origin_cm = 0;     if (!get_alt_cm(AltFrame::ABOVE_ORIGIN, alt_above_origin_cm))     {         return false;     }      // convert lat, lon     if (!get_vector_xy_from_origin_NE_cm(vec_neu.xy())) { </pre>                                                                                                                                                                                                                                                                                                                                                           | Geodetic (lat/lng/alt) → NEU centimetre vector relative to the EKF origin (geodetic↔NED duality).                                                                                                              |
| 22 | libraries/AP_InertialNav/AP_InertialNav.cpp    | 14-23   | AP_InertialNav::update()                  | Transform | <pre> void AP_InertialNav::update(bool high_vibes) {     // get the NE position relative to the local earth frame origin     Vector2f posNE;     if (_ahrs_ekf.get_relative_position_NE_origin_float(posNE)) {         _relpos_cm.x = posNE.x * 100; // convert from m to cm         _relpos_cm.y = posNE.y * 100; // convert from m to cm     }      // get the D position relative to the local earth frame origin </pre>                                                                                                                                                                                                                                                                     | Pulls EKF origin-relative NE/D position into _relpos_cm (cm). get_position_neu_cm() exposes it to vehicle code.                                                                                                |
| 23 | libraries/AC_AttitudeControl/AC_PosControl.cpp | 363-372 | AC_PosControl::input_pos_NEU_cm()         | Sink      | <pre> void AC_PosControl::input_pos_NEU_cm(const Vector3p&amp; pos_neu_cm, float pos_terrain_target_u_cm, float terrain_buffer_cm) {     input_pos_NEU_m(pos_neu_cm * 0.01, pos_terrain_target_u_cm * 0.01, terrain_buffer_cm * 0.01); }  // Sets a new NEU position target in meters and computes a jerk-limited trajectory. // Updates internal acceleration commands using a smooth kinematic path constrained // by configured acceleration and jerk limits. Terrain margin is used to constrain // horizontal velocity to avoid vertical buffer violation. void AC_PosControl::input_pos_NEU_m(const Vector3p&amp; pos_neu_m, float pos_terrain_target_u_m, float terrain_buffer_m) </pre> | 3D position/velocity controller consuming the position target. [ABSENT] there is no standalone AC_PosControl library — it physically resides under libraries/AC_AttitudeControl/ (snapshot-fidelity note).     |

| #  | File Path                                 | Line(s) | Function / Class                      | Role   | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Notes                                                                                                                                                                             |
|----|-------------------------------------------|---------|---------------------------------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 24 | libraries/APM_Control/AR_PosControl.cpp   | 114-123 | AR_PosControl::update()               | Sink   | <pre> void AR_PosControl::update(float dt) {     // exit immediately if no current location, destination or dis     armed     Vector3p curr_pos_NED_m;     Vector3f curr_vel_NED;     if (!hal.util-&gt;get_soft_armed())    !AP::ahrs().get_relative_po     sition_NED_origin(curr_pos_NED_m)        !AP::ahrs().get_velocity_NED(curr_vel_NED)) {         _desired_speed = _atc.get_desired_speed_accel_limited(0.0f         , dt);         _desired_lat_accel = 0.0f;         _desired_turn_rate_rads = 0.0f; </pre> | Rover position controller; reads AP::ahrs().get_relative_position_NED_origin() and get_velocity_NED() (Shared global/singleton). Separate controller from Copter's AC_PosControl. |
| 25 | libraries/AP_Beacon/AP_Beacon.cpp         | 175-184 | AP_Beacon::get_vehicle_position_ned() | Source | <pre> bool AP_Beacon::get_vehicle_position_ned(Vector3f &amp;position, float &amp; accuracy_estimate) const {     if (!device_ready()) {         return false;     }      // check for timeout     if (AP_HAL::millis() - veh_pos_update_ms &gt; AP_BEACON_TIMEOUT_M S) {         return false;     } </pre>                                                                                                                                                                                                            | Non-GPS (GPS-denied) positioning source; returns vehicle NED position with a staleness timeout.                                                                                   |
| 26 | libraries/AP_VisualOdom/AP_VisualOdom.cpp | 201-210 | AP_VisualOdom::handle_pose_estimate() | Source | <pre> void AP_VisualOdom::handle_pose_estimate(uint64_t remote_time_us, uint32_t time_ms, float x, float y, float z, float roll, float pit ch, float yaw, float posErr, float angErr, uint8_t reset_counter, int8_t quality) {     // exit immediately if not enabled     if (!enabled()) {         return;     }      // call backend     if (_driver != nullptr) {         // convert attitude to quaternion and call backend </pre>                                                                                  | Non-GPS visual-odometry position/attitude source feeding EKF external-nav fusion.                                                                                                 |

Table 1a — Positioning dependencies (Layer 1 direct edges + Layer 2 transitive chains)

## Layer 1 — Direct calls / called-by with typed dependency

| Ref # | Component / Function | Directly Calls                                          | Directly Called By                | Dependency Type         | Code Snippet                                                                                                                                                                                                                                                                                                                                                                           |
|-------|----------------------|---------------------------------------------------------|-----------------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | AP_GPS_UBLOX backend | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read() | Inheritance / Interface | <pre> _last_pos_time = _buffer.posllh.itow; state.location.lng = _buffer.posllh.longitude; state.location.lat = _buffer.posllh.latitude; state.have_undulation = true; state.undulation = (_buffer.posllh.altitude_msl - _buffer. posllh.altitude_ellipsoid) * 0.001; set_alt_amsl_cm(state, _buffer.posllh.altitude_msl / 10);  state.status = next_fix; _new_position = true; </pre> |
| 2     | AP_GPS_NMEA backend  | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read() | Inheritance / Interface | <pre> state.location.lat = _new_latitude; state.location.lng = _new_longitude; } state.num_sats = _new_satellite_count; state.hdop = _new_hdop; switch(_new_quality_indicator) { case 0: // Fix not available or invalid state.status = AP_GPS::NO_FIX; </pre>                                                                                                                         |

| Ref # | Component / Function | Directly Calls                                          | Directly Called By                | Dependency Type         | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                |
|-------|----------------------|---------------------------------------------------------|-----------------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3     | AP_GPS_SBF backend   | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read() | Inheritance / Interface | <pre> state.location.lat = (int32_t)(temp.Latitude * RAD_TO_DEG_DOUBLE * (double)1e7); state.location.lng = (int32_t)(temp.Longitude * RAD_TO_DEG_DOUBLE * (double)1e7); state.have_undulation = !is_DNU(temp.Undulation); double height = temp.Height; // in metres if (state.have_undulation) {     height -= temp.Undulation;     state.undulation = -temp.Undulation; } </pre>                          |
| 4     | AP_GPS_NOVA backend  | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read() | Inheritance / Interface | <pre> state.last_gps_time_ms = AP_HAL::millis();  state.location.lat = (int32_t) (bestposu.lat * (double)1e7); state.location.lng = (int32_t) (bestposu.lng * (double)1e7);  state.have_undulation = true; state.undulation = -bestposu.undulation; set_alt_amsl_cm(state, bestposu.hgt * 100); </pre>                                                                                                      |
| 5     | AP_GPS_SBP backend   | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read() | Inheritance / Interface | <pre> state.location.lat = (int32_t) (pos_llh-&gt;lat * (double)1e7); state.location.lng = (int32_t) (pos_llh-&gt;lon * (double)1e7); state.location.alt = (int32_t) (pos_llh-&gt;height * 100); state.num_sats = pos_llh-&gt;n_sats;  if (pos_llh-&gt;flags == 0) {     state.status = AP_GPS::GPS_OK_FIX_3D; } </pre>                                                                                     |
| 6     | AP_GPS_SBP2 backend  | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read() | Inheritance / Interface | <pre> // state.location.lat = (int32_t) (last_pos_llh.lat * (double)1e7); state.location.lng = (int32_t) (last_pos_llh.lon * (double)1e7); state.location.alt = (int32_t) (last_pos_llh.height * 100); state.num_sats = last_pos_llh.n_sats;  switch (last_pos_llh.flags.fix_mode) {     case 1: </pre>                                                                                                     |
| 7     | AP_GPS_SIRF backend  | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read() | Inheritance / Interface | <pre> state.status = AP_GPS::NO_FIX; }else if ((_buffer.nav.fix_type &amp; FIX_MASK) == FIX_3D) {     state.status = AP_GPS::GPS_OK_FIX_3D; }else{     state.status = AP_GPS::GPS_OK_FIX_2D; } state.location.lat = int32_t(be32toh(_buffer.nav.latitude)); state.location.lng = int32_t(be32toh(_buffer.nav.longitude)); </pre>                                                                            |
| 8     | AP_GPS_ERB backend   | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read() | Inheritance / Interface | <pre> _last_pos_time = _buffer.pos.time; state.location.lng = (int32_t)(_buffer.pos.longitude * (double)1e7); state.location.lat = (int32_t)(_buffer.pos.latitude * (double)1e7); state.have_undulation = true; state.undulation = _buffer.pos.altitude_msl - _buffer.pos.altitude_ellipsoid; set_alt_amsl_cm(state, _buffer.pos.altitude_msl * 100); state.status = next_fix; _new_position = true; </pre> |

| Ref # | Component / Function               | Directly Calls                                          | Directly Called By                             | Dependency Type         | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------|------------------------------------|---------------------------------------------------------|------------------------------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 9     | AP_GPS_MAV backend                 | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read()              | Inheritance / Interface | <pre> state.status = (AP_GPS::GPS_Status)packet.fix_type;  Location loc = {}; loc.lat = packet.lat; loc.lng = packet.lon; if (have_alt) {     loc.alt = packet.alt * 100; // convert to centimeters } state.location = loc; </pre>                                                                                                                                                                                                                          |
| 10    | AP_GPS_MSP backend                 | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read()              | Inheritance / Interface | <pre> state.status = (AP_GPS::GPS_Status)pkt.fix_type; state.num_sats = pkt.satellites_in_view; state.location = {     pkt.latitude,     pkt.longitude,     pkt.msl_altitude,     Location::AltFrame::ABSOLUTE }; </pre>                                                                                                                                                                                                                                    |
| 11    | AP_GPS_GSOFF backend               | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read()              | Inheritance / Interface | <pre> state.location.lat = (int32_t)(RAD_TO_DEG_DOUBLE * position.latitude_rad * (double)1e7); state.location.lng = (int32_t)(RAD_TO_DEG_DOUBLE * position.longitude_rad * (double)1e7); state.location.alt = (int32_t)(position.altitude * 100); state.last_gps_time_ms = AP_HAL::millis();  if ((vel.velocity_flags &amp; 1) == 1) {     state.ground_speed = vel.horizontal_velocity;     state.ground_course = wrap_360(degrees(vel.heading)); } </pre> |
| 12    | AP_GPS_DroneCAN backend            | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read()              | Inheritance / Interface | <pre> if (msg.height_msl_mm != msg.height_ellipsoid_mm) {     seen_valid_height_ellipsoid = true; } interim_state.have_undulation = seen_valid_height_ellipsoid;  if (seen_valid_height_ellipsoid) {     interim_state.undulation = (msg.height_msl_mm - msg.height_ellipsoid_mm) * 0.001; } interim_state.location = loc; set_alt_amsl_cm(interim_state, alt_amsl_cm); </pre>                                                                              |
| 13    | AP_GPS_ExternalAHRS backend        | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read()              | Inheritance / Interface | <pre> state.num_sats = pkt.satellites_in_view;  const Location loc {     pkt.latitude,     pkt.longitude,     pkt.msl_altitude,     Location::AltFrame::ABSOLUTE }; </pre>                                                                                                                                                                                                                                                                                  |
| 14    | AP_GPS_SITL backend                | writes GPS_State.location/status;<br>fill_3d_velocity() | AP_GPS::update() → backend read()              | Inheritance / Interface | <pre> state.status = AP_GPS::GPS_OK_FIX_3D; state.num_sats = 15;  state.location = Location{     int32_t(latitude*1e7),     int32_t(longitude*1e7),     int32_t(altitude*100),     Location::AltFrame::ABSOLUTE }; </pre>                                                                                                                                                                                                                                   |
| 15    | AP_GPS_Backend::fill_3d_velocity() | cosf/sinf on state.ground_course; writes state.velocity | All receiver backends (UBLOX, NMEA, SIRF, ...) | Data dependency         | <pre> void AP_GPS_Backend::fill_3d_velocity(void) {     float gps_heading = radians(state.ground_course);      state.velocity.x = state.ground_speed * cosf(gps_heading);     state.velocity.y = state.ground_speed * sinf(gps_heading);     state.velocity.z = 0;     state.have_vertical_velocity = false; } </pre>                                                                                                                                       |



| Ref # | Component / Function                      | Directly Calls                                     | Directly Called By                                       | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------|-------------------------------------------|----------------------------------------------------|----------------------------------------------------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 16    | AP_GPS::location()                        | returns state[instance].location                   | NavEKF3_Measurements::readGpsData(); AP_AHRS; AC_WPNNav  | Function call   | <pre> const Location &amp;location(uint8_t instance) const {     return state[instance].location; } const Location &amp;location() const {     return location(primary_instance); } </pre>                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 17    | AP_GPS::velocity()                        | returns state[instance].velocity                   | NavEKF3_Measurements::readGpsData() (PosVel fusion)      | Function call   | <pre> const Vector3f &amp;velocity(uint8_t instance) const {     return state[instance].velocity; } const Vector3f &amp;velocity() const {     return velocity(primary_instance); } </pre>                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 18    | AP_GPS::GPS_State (struct)                | (data struct — no calls)                           | All GPS backends (write); frontend accessors (read)      | Data dependency | <pre> struct GPS_State {     uint8_t instance; // the instance number of this GPS      // all the following fields must all be filled by the back end driver     GPS_Status status;           ///&lt; driver fix status     uint32_t time_week_ms;       ///&lt; GPS time (millise conds from start of GPS week)     uint16_t time_week;          ///&lt; GPS week number     Location location;           ///&lt; last fix location     float ground_speed;          ///&lt; ground speed in m /s     float ground_course;         ///&lt; ground course in degrees, wrapped 0-360 </pre>                               |
| 19    | AP_GPS_Blended::_calc_weights()           | gps.state[].status; time deltas                    | AP_GPS_Blended::_calc_weights()/update (blended primary) | Data dependency | <pre> bool AP_GPS_Blended::_calc_weights(void) {     // zero the blend weights     memset(&amp;blend_weights, 0, sizeof(_blend_weights));      static_assert(GPS_MAX_RECEIVERS == 2, "GPS blending only curre ntly works with 2 receivers");     // Note that the early quit below relies upon exactly 2 instan ces     // The time delta calculations below also rely upon every inst ance being currently detected and being parsed </pre>                                                                                                                                                                             |
| 20    | class Location                            | get_alt_cm();<br>get_vector_xy_from_origin_NE_cm() | GPS_State; AP_AHRS; AC_WPNNav; AP_Mission (ubiquitous)   | Data dependency | <pre> uint8_t relative_alt : 1;           // 1 if altitude is relati ve to home uint8_t loiter_ccw    : 1;           // 0 if clockwise, 1 if co unter clockwise uint8_t terrain_alt   : 1;           // this altitude is above terrain uint8_t origin_alt    : 1;           // this altitude is above ekf origin uint8_t loiter_xtrack : 1;           // 0 to crosstrack from ce nter of waypoint, 1 to crosstrack from tangent exit location      // note that mission storage only stores 24 bits of altitude ( ~ +/- 83km) int32_t alt; // in cm int32_t lat; // in 1E7 degrees int32_t lng; // in 1E7 degrees </pre> |
| 21    | Location::get_vector_from_origin_NEU_cm() | get_alt_cm();<br>get_vector_xy_from_origin_NE_cm() | AC_WPNNav::get_vector_NEU_cm()                           | Function call   | <pre> bool Location::get_vector_from_origin_NEU_cm(T &amp;vec_neu) const {     // convert altitude     int32_t alt_above_origin_cm = 0;     if (!get_alt_cm(AltFrame::ABOVE_ORIGIN, alt_above_origin_cm))     {         return false;     }      // convert lat, lon     if (!get_vector_xy_from_origin_NE_cm(vec_neu.xy())) { </pre>                                                                                                                                                                                                                                                                                    |

| Ref # | Component / Function                  | Directly Calls                                                    | Directly Called By                  | Dependency Type           | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------|---------------------------------------|-------------------------------------------------------------------|-------------------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 22    | AP_InertialNav::update()              | _ahrs_ekf.get_relative_position_NE_origin_float()                 | Copter::read_inertia()              | Function call             | <pre> void AP_InertialNav::update(bool high_vibes) {     // get the NE position relative to the local earth frame origin     Vector2f posNE;     if (_ahrs_ekf.get_relative_position_NE_origin_float(posNE)) {         _relpos_cm.x = posNE.x * 100; // convert from m to cm         _relpos_cm.y = posNE.y * 100; // convert from m to cm     }      // get the D position relative to the local earth frame origin </pre>                                                                                                                                                                                                                                                                     |
| 23    | AC_PosControl::input_pos_NEU_cm()     | input_pos_NEU_m()                                                 | ArduCopter mode_auto.cpp / mode.cpp | Function call             | <pre> void AC_PosControl::input_pos_NEU_cm(const Vector3p&amp; pos_neu_cm, float pos_terrain_target_u_cm, float terrain_buffer_cm) {     input_pos_NEU_m(pos_neu_cm * 0.01, pos_terrain_target_u_cm * 0.01, terrain_buffer_cm * 0.01); }  // Sets a new NEU position target in meters and computes a jerk-limited trajectory. // Updates internal acceleration commands using a smooth kinematic path constrained // by configured acceleration and jerk limits. Terrain margin is used to constrain // horizontal velocity to avoid vertical buffer violation. void AC_PosControl::input_pos_NEU_m(const Vector3p&amp; pos_neu_m, float pos_terrain_target_u_m, float terrain_buffer_m) </pre> |
| 24    | AR_PosControl::update()               | AP::ahrs().get_relative_position_NED_origin(); get_velocity_NED() | Rover mode controllers              | Shared global / singleton | <pre> void AR_PosControl::update(float dt) {     // exit immediately if no current location, destination or disarmed     Vector3p curr_pos_NED_m;     Vector3f curr_vel_NED;     if (!hal.util-&gt;get_soft_armed()    !AP::ahrs().get_relative_position_NED_origin(curr_pos_NED_m)            !AP::ahrs().get_velocity_NED(curr_vel_NED)) {         _desired_speed = _atc.get_desired_speed_accel_limited(0.0f, dt);         _desired_lat_accel = 0.0f;         _desired_turn_rate_rads = 0.0f;     } </pre>                                                                                                                                                                                   |
| 25    | AP_Beacon::get_vehicle_position_ned() | device_ready(); AP_HAL::millis()                                  | NavEKF3 range-beacon fusion         | Function call             | <pre> bool AP_Beacon::get_vehicle_position_ned(Vector3f &amp;position, float &amp;accuracy_estimate) const {     if (!device_ready()) {         return false;     }      // check for timeout     if (AP_HAL::millis() - veh_pos_update_ms &gt; AP_BEACON_TIMEOUT_MS) {         return false;     } </pre>                                                                                                                                                                                                                                                                                                                                                                                      |
| 26    | AP_VisualOdom::handle_pose_estimate() | _driver backend handle_pose_estimate()                            | GCS / AP_ExternalAHRS pose handlers | Inheritance / Interface   | <pre> void AP_VisualOdom::handle_pose_estimate(uint64_t remote_time_us, uint32_t time_ms, float x, float y, float z, float roll, float pitch, float yaw, float posErr, float angErr, uint8_t reset_counter, int8_t quality) {     // exit immediately if not enabled     if (!enabled()) {         return;     }      // call backend     if (_driver != nullptr) {         // convert attitude to quaternion and call backend     } </pre>                                                                                                                                                                                                                                                     |

**Layer 2 — Full transitive chain to final consumer (hop count)**

| Ref # | PNT Origin                           | Full Dependency Chain                                                                                                                                                                                                             | Final Consumer          | Chain Depth | Notes                                                                                                      |
|-------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|-------------|------------------------------------------------------------------------------------------------------------|
| 1     | AP_GPS_UBLOX (state.location)        | AP_GPS_UBLOX → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors        | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 2     | AP_GPS_NMEA (state.location)         | AP_GPS_NMEA → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors         | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 3     | AP_GPS_SBF (state.location)          | AP_GPS_SBF → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors          | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 4     | AP_GPS_NOVA (state.location)         | AP_GPS_NOVA → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors         | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 5     | AP_GPS_SBP (state.location)          | AP_GPS_SBP → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors          | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 6     | AP_GPS_SBP2 (state.location)         | AP_GPS_SBP2 → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors         | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 7     | AP_GPS_SIRF (state.location)         | AP_GPS_SIRF → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors         | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 8     | AP_GPS_ERB (state.location)          | AP_GPS_ERB → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors          | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 9     | AP_GPS_MAV (state.location)          | AP_GPS_MAV → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors          | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 10    | AP_GPS_MSP (state.location)          | AP_GPS_MSP → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors          | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 11    | AP_GPS_GSOF (state.location)         | AP_GPS_GSOF → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors         | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 12    | AP_GPS_DroneCAN (state.location)     | AP_GPS_DroneCAN → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors     | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 13    | AP_GPS_ExternalAHRS (state.location) | AP_GPS_ExternalAHRS → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 14    | AP_GPS_SITL (state.location)         | AP_GPS_SITL → AP_GPS::location()/velocity() → NavEKF3_Measurements::readGpsData (gps.location/gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV → AC_PosControl → AC_AttitudeControl → AP_Motors         | AP_Motors / SRV_Channel | 8           | Sensor source to actuator; GPS ingested in AP_NavEKF3_Measurements.cpp (readGpsData) before FuseVelPosNED. |
| 15    | GPS velocity (fill_3d_velocity)      | GPS_State.velocity → AP_GPS::velocity() → NavEKF3_Measurements::readGpsData → NavEKF3::FuseVelPosNED → AP_AHRS::get_velocity_NED() → AC_PosControl → AP_Motors                                                                    | AP_Motors               | 6           | [MULTI-CATEGORY] also Navigation.                                                                          |
| 16    | AP_GPS::location()                   | AP_GPS::location() → NavEKF3_Measurements::readGpsData (gps.location) → NavEKF3::FuseVelPosNED → AP_AHRS::get_location() → AC_WPNV::set_wp_destination_loc → AC_PosControl → AP_Motors                                            | AP_Motors               | 6           | Geodetic position path (ingestion hop explicit).                                                           |

| Ref # | PNT Origin                           | Full Dependency Chain                                                                                                                                    | Final Consumer       | Chain Depth | Notes                                                                                                                                    |
|-------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------|------------------------------------------------------------------------------------------------------------------------------------------|
| 17    | AP_GPS::velocity()                   | AP_GPS::velocity() → NavEKF3_Measurements::readGpsData (gps.velocity) → NavEKF3::FuseVelPosNED → AP_AHRS::get_velocity_NED() → AC_PosControl → AP_Motors | AP_Motors            | 5           | Velocity path (ingestion hop explicit).                                                                                                  |
| 18    | GPS_State (struct)                   | GPS_State → AP_GPS accessors {location velocity time_week} → NavEKF3_Measurements → EKF/AHRS + AP_RTC → controllers + clock                              | AP_Motors and AP_RTC | 4           | [SHARED-STRUCT][MULTI-CATEGORY] branches into Table 3.                                                                                   |
| 19    | AP_GPS_Blended                       | Blended → AP_GPS primary (blended) location/velocity → NavEKF3_Measurements → NavEKF3::FuseVelPosNED → AP_AHRS → controllers → AP_Motors                 | AP_Motors            | 6           | Transform upstream of EKF ingestion.                                                                                                     |
| 20    | class Location (geodetic value type) | Location → AC_WPNav::set_wp_destination_loc → get_vector_NEU_cm → AC_PosControl → AC_AttitudeControl → AP_Motors                                         | AP_Motors            | 5           | [SHARED-STRUCT] Pervasive value type; depth measured along its dominant waypoint-control consumer branch (the AC_WPNav→PosControl path). |
| 21    | Location→NEU conversion              | get_vector_from_origin_NEU_cm() → AC_WPNav::get_vector_NEU_cm → set_wp_destination_NEU_cm → AC_PosControl                                                | AC_PosControl        | 3           | Geodetic↔NED transform edge.                                                                                                             |
| 22    | AP_InertialNav                       | EKF origin-relative pos → AP_InertialNav::update → _relpos_cm → Copter logging/reporting                                                                 | GCS / logger         | 3           | Reporting branch.                                                                                                                        |
| 23    | AC_PosControl (Sink)                 | AP_AHRS pos → AC_WPNav → AC_PosControl → AC_AttitudeControl → AP_Motors                                                                                  | AP_Motors            | 4           | Inbound chain to a Sink endpoint.                                                                                                        |
| 24    | AR_PosControl (Sink, Rover)          | AP_AHRS pos/vel → AR_PosControl → AR_AttitudeControl → SRV_Channels                                                                                      | SRV_Channels         | 3           | Rover actuation endpoint.                                                                                                                |
| 25    | AP_Beacon (Source)                   | AP_Beacon → get_vehicle_position_ned → NavEKF3 RngBcn fusion → AP_AHRS → AP_Motors                                                                       | AP_Motors            | 4           | GPS-denied source.                                                                                                                       |
| 26    | AP_VisualOdom (Source)               | AP_VisualOdom → NavEKF3 external-nav fusion → AP_AHRS → AP_Motors                                                                                        | AP_Motors            | 3           | GPS-denied source.                                                                                                                       |

Table 2 — Core Navigation (AHRS hub, EKF fusion, waypoint/mission nav, fixed-wing guidance &amp; energy, attitude sink)

| # | File Path                                | Line(s)   | Function / Class                            | Role      | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                       | Notes                                                                                                                                                                                                                                                              |
|---|------------------------------------------|-----------|---------------------------------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | libraries/AP_AHRS/AP_AHRS.cpp            | 3600-3604 | AP_AHRS::get_location()                     | Transform | <pre>bool AP_AHRS::get_location(Location &amp;loc) const {     loc = state.location;     return state.location_ok; }</pre>                                                                                                                                                                                                                                      | Central navigation hub returns the fused Location from the active EKF. Reached via AP::ahrs(). [CIRCULAR] AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference.                                    |
| 2 | libraries/AP_AHRS/AP_AHRS.h              | 238-246   | AP_AHRS::groundspeed()                      | Sink      | <pre>// EKF has a better ground speed vector estimate const Vector2f &amp;groundspeed_vector() const { return state.ground_speed_vec; }  // return ground speed estimate in meters/second. Used by ground vehicles. float groundspeed(void) const { return state.ground_speed; }  const Vector3f &amp;get_accel_ef() const {     return state.accel_ef; }</pre> | Read accessor returning the already-fused state.ground_speed (m/s). Performs no fusion and produces nothing from a measurement — classified Sink (read accessor), NOT Source.                                                                                      |
| 3 | libraries/AP_AHRS/AP_AHRS.h              | 269-277   | AP_AHRS::get_origin() / have_inertial_nav() | Sink      | <pre>// returns the inertial navigation origin in lat/lon/alt bool get_origin(Location &amp;ret) const WARN_IF_UNUSED;  bool have_inertial_nav() const;  // return a ground velocity in meters/second, North/East/Down // order. Must only be called if have_inertial_nav() is true bool get_velocity_NED(Vector3f &amp;vec) const WARN_IF_UNUSED;</pre>        | Availability/origin read accessors (have_inertial_nav()) is a user-cited symbol, L273). They report EKF availability/origin — read accessors, NOT measurement Sources; classified Sink. EKF origin is WRITTEN by the EKF (see #15 calcGpsGoodToAlign / setOrigin). |
| 4 | libraries/AP_AHRS/AP_AHRS.cpp            | 1739-1748 | AP_AHRS::get_relative_position_NED_origin() | Transform | <pre>bool AP_AHRS::get_relative_position_NED_origin(Vector3p &amp;vec) const {     switch (active_EKF_type()) {         #if AP_AHRS_DCM_ENABLED         case EKFTYPE::DCM:             return dcm.get_relative_position_NED_origin(vec);         #endif         #if HAL_NAV_EKF2_AVAILABLE         case EKFTYPE::TW0: {             Vector2p posNE; </pre>      | Geodetic↔NED: origin-relative NED position via active-EKF-type dispatch (DCM/EKF2/EKF3) — performs conversion on read.                                                                                                                                             |
| 5 | libraries/AP_AHRS/AP_AHRS.cpp            | 3662-3666 | AP_AHRS::get_velocity_NED()                 | Transform | <pre>bool AP_AHRS::get_velocity_NED(Vector3f &amp;vec) const {     vec = state.velocity_NED;     return state.velocity_NED_ok; }</pre>                                                                                                                                                                                                                          | Exposes the fused NED velocity (nav-hub output of EKF fusion); consumed by crash detection and Rover position control.                                                                                                                                             |
| 6 | libraries/AP_NavEKF3/AP_NavEKF3_core.cpp | 627-636   | NavEKF3_core::UpdateFilter()                | Transform | <pre>void NavEKF3_core::UpdateFilter(bool predict) {     // don't run filter updates if states have not been initialised     if (!statesInitialised) {         return;     }      fill_scratch_variables();      // update sensor selection (for affinity) </pre>                                                                                               | Extended Kalman Filter predict/fuse step (the core PNT transform). [CIRCULAR] AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference. One of 13 AP_NavEKF3 fusion modules, all catalogued.           |

| #  | File Path                                        | Line(s) | Function / Class                   | Role      | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Notes                                                                                                                                                                                                                                                                                                                                                    |
|----|--------------------------------------------------|---------|------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7  | libraries/AP_NavEKF3/AP_NavEKF3_PosVelFusion.cpp | 502-511 | NavEKF3_core::SelectVelPosFusion() | Transform | <pre> void NavEKF3_core::SelectVelPosFusion() {     // Check if the magnetometer has been fused on that time step     and the filter is running at faster than 200 Hz     // If so, don't fuse measurements on this time step to reduce     frame over-runs     // Only allow one time slip to prevent high rate magnetometer     data preventing fusion of other measurements     if (magFusePerformed &amp;&amp; dtIMUavg &lt; 0.005f &amp;&amp; !posVelFusionDelayed) {         posVelFusionDelayed = true;         return;     } else {         posVelFusionDelayed = false;     } } </pre> | GPS position/velocity fusion scheduling; the dtIMUavg<0.005 gate couples EKF fusion cadence to the scheduler loop rate (timing coupling). [CIRCULAR] AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference. One of 13 AP_NavEKF3 fusion modules, all catalogued.                          |
| 8  | libraries/AP_NavEKF3/AP_NavEKF3_PosVelFusion.cpp | 694-702 | NavEKF3_core::FuseVelPosNED()      | Transform | <pre> void NavEKF3_core::FuseVelPosNED() {     // declare variables used to control access to arrays     bool fuseData[6] {};     uint8_t stateIndex;     uint8_t obsIndex;      // declare variables used by state and covariance update calculations     Vector6 R_OBS; // Measurement variances used for fusion } </pre>                                                                                                                                                                                                                                                                     | Core position/velocity measurement-update (Kalman gain applied to GPS/baro NED innovations). One of 13 AP_NavEKF3 fusion modules, all catalogued.                                                                                                                                                                                                        |
| 9  | libraries/AP_NavEKF3/AP_NavEKF3_Measurements.cpp | 559-567 | NavEKF3_core::readGpsData()        | Transform | <pre> void NavEKF3_core::readGpsData() {     // check for new GPS data     const auto &amp;gps = dal.gps();      // limit update rate to avoid overflowing the FIFO buffer     if (gps.last_message_time_ms(selected_gps) - lastTimeGpsReceived_ms &lt;= frontend-&gt;sensorIntervalMin_ms) {         return;     } } </pre>                                                                                                                                                                                                                                                                    | GPS ingestion into the EKF delayed-time buffer (reads gps.location()/velocity()/status via AP_DAL). This is the GPS→EKF hop omitted earlier (cf. Table 1a chains). [CIRCULAR] AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference. One of 13 AP_NavEKF3 fusion modules, all catalogued. |
| 10 | libraries/AP_NavEKF3/AP_NavEKF3_Control.cpp      | 232-240 | NavEKF3_core::setAidingMode()      | Transform | <pre> void NavEKF3_core::setAidingMode() {     resetDataSource posResetSource = resetDataSource::DEFAULT;     resetDataSource velResetSource = resetDataSource::DEFAULT;      // Save the previous status so we can detect when it has changed     PV_AidingModePrev = PV_AidingMode;      setYawSource(); } </pre>                                                                                                                                                                                                                                                                             | Navigation aiding-mode state machine (none/relative/absolute) driving which PNT measurements are fused. One of 13 AP_NavEKF3 fusion modules, all catalogued.                                                                                                                                                                                             |
| 11 | libraries/AP_NavEKF3/AP_NavEKF3_MagFusion.cpp    | 227-235 | NavEKF3_core::SelectMagFusion()    | Transform | <pre> void NavEKF3_core::SelectMagFusion() {     // clear the flag that lets other processes know that the expensive magnetometer fusion operation has been performed on that time step     // used for load levelling     magFusePerformed = false;      // Store yaw angle when moving for use as a static reference when not moving     if (!onGroundNotMoving) {         if (fabsF(prevTnb[0][2]) &lt; fabsF(prevTnb[1][2])) { </pre>                                                                                                                                                       | Magnetometer/yaw fusion scheduling feeding the heading component of the navigation solution. One of 13 AP_NavEKF3 fusion modules, all catalogued.                                                                                                                                                                                                        |

| #  | File Path                                         | Line(s) | Function / Class                   | Role      | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Notes                                                                                                                                               |
|----|---------------------------------------------------|---------|------------------------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 12 | libraries/AP_NavEKF3/AP_NavEKF3_AirDataFusion.cpp | 193-201 | NavEKF3_core::SelectTasFusion()    | Transform | <pre>void NavEKF3_core::SelectTasFusion() {     // Check if the magnetometer has been fused on that time step     and the filter is running at faster than 200 Hz     // If so, don't fuse measurements on this time step to reduce     frame over-runs     // Only allow one time slip to prevent high rate magnetometer     data locking out fusion of other measurements     if (magFusePerformed &amp;&amp; dtIMUavg &lt; 0.005f &amp;&amp; !airSpdFusionDelayed) {         airSpdFusionDelayed = true;         return;     } else {</pre>   | True-airspeed / synthetic-sideslip fusion (wind/velocity estimation for fixed-wing nav). One of 13 AP_NavEKF3 fusion modules, all catalogued.       |
| 13 | libraries/AP_NavEKF3/AP_NavEKF3_OptFlowFusion.cpp | 21-29   | NavEKF3_core::SelectFlowFusion()   | Transform | <pre>void NavEKF3_core::SelectFlowFusion() {     // Check if the magnetometer has been fused on that time step     and the filter is running at faster than 200 Hz     // If so, don't fuse measurements on this time step to reduce     frame over-runs     // Only allow one time slip to prevent high rate magnetometer     data preventing fusion of other measurements     if (magFusePerformed &amp;&amp; dtIMUavg &lt; 0.005f &amp;&amp; !optFlowFusionDelayed) {         optFlowFusionDelayed = true;         return;     } else {</pre> | Optical-flow fusion scheduling for GPS-denied velocity/position aiding. One of 13 AP_NavEKF3 fusion modules, all catalogued.                        |
| 14 | libraries/AP_NavEKF3/AP_NavEKF3_Outputs.cpp       | 326-334 | NavEKF3_core::getLLH()             | Transform | <pre>bool NavEKF3_core::getLLH(Location &amp;loc) const {     Location origin;     if (getOriginLLH(origin)) {         postype_t posD;         if (getPosD_local(posD) &amp;&amp; PV_AidingMode != AID_NONE) {             // Altitude returned is an absolute altitude relative             to the WGS-84 spherioid             loc.set_alt_cm(origin.alt - posD*100.0, Location::AltFrame::ABSOLUTE);             if (filterStatus.flags.horiz_pos_abs    filterStatus.flags.horiz_pos_rel) {</pre>                                            | EKF position output: converts the filter state to a geodetic Location consumed by AP_AHRS. One of 13 AP_NavEKF3 fusion modules, all catalogued.     |
| 15 | libraries/AP_NavEKF3/AP_NavEKF3_VehicleStatus.cpp | 14-22   | NavEKF3_core::calcGpsGoodToAlign() | Transform | <pre>void NavEKF3_core::calcGpsGoodToAlign(void) {     if (inFlight &amp;&amp; !finalInflightYawInit &amp;&amp; assume_zero_sideslip() &amp;&amp; !use_compass()) {         // this is a special case where a plane has launched without magnetometer         // is now in the air and needs to align yaw to the GPS and         start navigating as soon as possible         gpsGoodToAlign = true;         return;     }</pre>                                                                                                                 | GPS-quality gate deciding whether the GPS is good enough to align/aid the navigation solution. One of 13 AP_NavEKF3 fusion modules, all catalogued. |
| 16 | libraries/AP_NavEKF3/AP_NavEKF3_GyroBias.cpp      | 6-13    | NavEKF3_core::resetGyroBias()      | Transform | <pre>void NavEKF3_core::resetGyroBias(void) {     stateStruct.gyro_bias.zero();     zeroRows(P,10,12);     zeroCols(P,10,12);      P[10][10] = sq(radians(0.5f * dtIMUavg));     P[11][11] = P[10][10];</pre>                                                                                                                                                                                                                                                                                                                                    | Gyro-bias state reset within the inertial-navigation state vector. One of 13 AP_NavEKF3 fusion modules, all catalogued.                             |
| 17 | libraries/AP_NavEKF3/AP_NavEKF3_RngBcnFusion.cpp  | 95-103  | NavEKF3_core::FuseRngBcn()         | Transform | <pre>void NavEKF3_core::FuseRngBcn() {     // declarations     ftype pn;     ftype pe;     ftype pd;     ftype bcn_pn;     ftype bcn_pe;     ftype bcn_pd;</pre>                                                                                                                                                                                                                                                                                                                                                                                 | Range-beacon (non-GPS) position fusion — consumes AP_Beacon ranges (Table 1 #25). One of 13 AP_NavEKF3 fusion modules, all catalogued.              |

| #  | File Path                                        | Line(s)   | Function / Class                   | Role      | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Notes                                                                                                                                                                                                                                                                               |
|----|--------------------------------------------------|-----------|------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 18 | libraries/AP_NavEKF3/AP_NavEKF3_Loading.cpp      | 15-23     | NavEKF3_core::Log_Write_XKF1()     | Sink      | <pre> void NavEKF3_core::Log_Write_XKF1(uint64_t time_us) const {     // Write first EKF packet     Vector3f euler;     Vector2p posNE;     postype_t posD;     Vector3f velNED;     Vector3f gyroBias;     float posDownDeriv; </pre>                                                                                                                                                                                                                                                                                 | Logs the fused navigation state (pos/vel/attitude) to the dataflash; reads PNT state, drives no control — a logging Sink. One of 13 AP_NavEKF3 fusion modules, all catalogued.                                                                                                      |
| 19 | libraries/AP_NavEKF3/AP_NavEKF3.cpp              | 1846-1854 | NavEKF3::getFilterStatus()         | Transform | <pre> void NavEKF3::getFilterStatus(nav_filter_status &amp;status) const {     if (core) {         core[primary].getFilterStatus(status);     } else {         memset(&amp;status, 0, sizeof(status));     } } </pre>                                                                                                                                                                                                                                                                                                  | EKF3 frontend: exposes the per-core nav_filter_status (position/velocity validity bits) to vehicle health logic (Group 2).                                                                                                                                                          |
| 20 | libraries/AP_NavEKF2/AP_NavEKF2_core.cpp         | 528-536   | NavEKF2_core::UpdateFilter()       | Transform | <pre> void NavEKF2_core::UpdateFilter(bool predict) {     // Set the flag to indicate to the filter that the front-end has     // as given permission for a new state prediction cycle to be started     startPredictEnabled = predict;      // don't run filter updates if states have not been initialised     if (!statesInitialised) {         return;     } } </pre>                                                                                                                                              | Second-generation EKF predict/fuse step; selectable alternative to EKF3. [CIRCULAR] AHRS owns NavEKF2/NavEKF3 while the EKF core reads AHRS/sensor state back through AP_DAL — AHRS↔EKF mutual reference. [DUPLICATED] parallel EKF implementation to AP_NavEKF3 (extraction note). |
| 21 | libraries/AP_NavEKF2/AP_NavEKF2_PosVelFusion.cpp | 560-568   | NavEKF2_core::FuseVelPosNED()      | Transform | <pre> void NavEKF2_core::FuseVelPosNED() {     // health is set bad until test passed     bool velHealth = false; // boolean true if velocity measurements have passed innovation consistency check     bool posHealth = false; // boolean true if position measurements have passed innovation consistency check     bool hgtHealth = false; // boolean true if height measurements have passed innovation consistency check      // declare variables used to check measurement errors     Vector3F velInnov; </pre> | EKF2 GPS position/velocity measurement-update; structurally mirrors EKF3 FuseVelPosNED (#8). [DUPLICATED].                                                                                                                                                                          |
| 22 | libraries/AP_NavEKF/EKFGSF_yaw.cpp               | 30-38     | EKFGSF_yaw::update()               | Transform | <pre> void EKFGSF_yaw::update(const Vector3F &amp;delAng,                         const Vector3F &amp;delVel,                         const ftype delAngDT,                         const ftype delVelDT,                         bool runEKF,                         ftype TAS) {     // copy to class variables </pre>                                                                                                                                                                                              | Gaussian-Sum-Filter emergency yaw estimator (shared by EKF2/EKF3) providing a GPS-velocity-derived yaw used to recover the navigation solution.                                                                                                                                     |
| 23 | libraries/AC_WPNav/AC_WPNav.cpp                  | 245-254   | AC_WPNav::set_wp_destination_loc() | Transform | <pre> bool AC_WPNav::set_wp_destination_loc(const Location&amp; destination) {     bool is_terrain_alt;     Vector3f dest_neu_cm;      // convert destination location to vector     if (!get_vector_NEU_cm(destination, dest_neu_cm, is_terrain_alt)) {         return false;     } } </pre>                                                                                                                                                                                                                          | Waypoint navigation: converts a Location waypoint to a NEU target (get_vector_NEU_cm) relative to EKF origin.                                                                                                                                                                       |



| #  | File Path                                           | Line(s)   | Function / Class                                          | Role      | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Notes                                                                                                                                                                                                            |
|----|-----------------------------------------------------|-----------|-----------------------------------------------------------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 24 | libraries/AP_Mission/AP_Mission.cpp                 | 552-561   | AP_Mission::get_next_nav_cmd()                            | Transform | <pre> bool AP_Mission::get_next_nav_cmd(uint16_t start_index, Mission_Command&amp; cmd) {     // search until the end of the mission command list     for (uint16_t cmd_index = start_index; cmd_index &lt; (unsigned)_cmd_total; cmd_index++) {         // get next command         if (!get_next_cmd(cmd_index, cmd, false)) {             // no more commands so return failure             return false;         }         // if found a "navigation" command then return it </pre>                                                                                                                                                                       | Mission-command storage reader + navigation-command sequencing: scans stored commands and returns the next NAV waypoint. NOT a PNT measurement Source — it sequences stored navigation targets (Transform).      |
| 25 | libraries/AP_L1_Control/AP_L1_Control.cpp           | 226-234   | AP_L1_Control::update_waypoint()                          | Transform | <pre> // Calculate L1 gain required for specified damping float K_L1 = 4.0f * _L1_damping * _L1_damping;  // Get current position and velocity if (!_ahrs.get_location(_current_loc) == false) {     // if no GPS loc available, maintain last nav/target bearing     _data_is_stale = true;     return; } </pre>                                                                                                                                                                                                                                                                                                                                             | Fixed-wing lateral (L1) guidance consuming the AHRS navigation position via _ahrs.get_location() (returns early if unavailable). The same method also reads AP_HAL::micros() at L214 for the dt timing coupling. |
| 26 | libraries/AP_L1_Control/AP_L1_Control.cpp           | 79-88     | AP_L1_Control::nav_roll_cd()                              | Transform | <pre> int32_t AP_L1_Control::nav_roll_cd(void) const {     float ret;     /*      * formula can be obtained through equations of balanced spiral:      * liftForce * cos(roll) = gravityForce * cos(pitch);      * liftForce * sin(roll) = gravityForce * lateralAcceleration / gravityAcceleration; // as mass = gravityForce/gravityAcceleration      * see issue 24319 [https://github.com/ArduPilot/ardupilot/issues/24319]      * Multiplier 100.0f is for converting degrees to centidegrees      * Made changes to avoid zero division as proposed by Andrew Tridgell: https://github.com/ArduPilot/ardupilot/pull/24331#discussion_r1267798397 </pre> | Derives demanded roll (centi-deg) from the lateral-acceleration demand and AHRS pitch.                                                                                                                           |
| 27 | libraries/AP_TECS/AP_TECS.cpp                       | 1259-1268 | AP_TECS::update_pitch_throttle()                          | Transform | <pre> void AP_TECS::update_pitch_throttle(int32_t hgt_dem_cm,                                    int32_t EAS_dem_cm,                                    enum AP_FixedWing::FlightStage flight_stage,                                    float distance_beyond_land_wp,                                    int32_t ptchMinCO_cd,                                    int16_t throttle_nudge,                                    float hgt_afe,                                    float load_factor,                                    float pitch_trim_deg) { </pre>                                                                                          | Fixed-wing Total Energy Control (speed/altitude) producing pitch & throttle demands.                                                                                                                             |
| 28 | libraries/AC_AttitudeControl/AC_AttitudeControl.cpp | 411-419   | AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd() | Sink      | <pre> void AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd(float euler_roll_angle_cd, float euler_pitch_angle_cd, float euler_yaw_angle_cd, bool slew_yaw) {     // Convert from centidegrees on public interface to radians     const float euler_roll_angle_rad = cd_to_rad(euler_roll_angle_cd);     const float euler_pitch_angle_rad = cd_to_rad(euler_pitch_angle_cd);     const float euler_yaw_angle_rad = cd_to_rad(euler_yaw_angle_cd);      input_euler_angle_roll_pitch_yaw_rad(euler_roll_angle_rad, euler_pitch_angle_rad, euler_yaw_angle_rad, slew_yaw); } </pre>                                                                     | Attitude/rate controller consuming the navigation demand; downstream of all navigation.                                                                                                                          |

Table 2a — Navigation dependencies (Layer 1 direct edges + Layer 2 transitive chains)

## Layer 1 — Direct calls / called-by with typed dependency

| Ref # | Component / Function                        | Directly Calls                                                    | Directly Called By                                           | Dependency Type         | Code Snippet                                                                                                                                                                                                                                                                                                                                                    |
|-------|---------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | AP_AHRS::get_location()                     | returns state.location (fused by active EKF)                      | ArduCopter inertia.cpp; commands.cpp; GCS_MAVLink_Copter.cpp | Function call           | <pre>bool AP_AHRS::get_location(Location &amp;loc) const {     loc = state.location;     return state.location_ok; }</pre>                                                                                                                                                                                                                                      |
| 2     | AP_AHRS::groundspeed()                      | returns state.ground_speed                                        | Rover speed control; GCS reporting                           | Data dependency         | <pre>// EKF has a better ground speed vector estimate const Vector2f &amp;groundspeed_vector() const { return state.ground_speed_vec; }  // return ground speed estimate in meters/second. Used by ground vehicles. float groundspeed(void) const { return state.ground_speed; }  const Vector3f &amp;get_accel_ef() const {     return state.accel_ef; }</pre> |
| 3     | AP_AHRS::have_inertial_nav()/get_origin()   | active_EKF_type(); EKF origin                                     | Copter::ekf_has_absolute_position (system.cpp); ekf_check    | Function call           | <pre>// returns the inertial navigation origin in lat/lon/alt bool get_origin(Location &amp;ret) const WARN_IF_UNUSED;  bool have_inertial_nav() const;  // return a ground velocity in meters/second, North/East/Down // order. Must only be called if have_inertial_nav() is true bool get_velocity_NED(Vector3f &amp;vec) const WARN_IF_UNUSED;</pre>        |
| 4     | AP_AHRS::get_relative_position_NED_origin() | dcm/EKF2/EKF3<br>get_relative_position_NED_origin()               | AR_PosControl::update;<br>AP_InertialNav::update             | Inheritance / Interface | <pre>bool AP_AHRS::get_relative_position_NED_origin(Vector3p &amp;vec) const {     switch (active_EKF_type()) {     #if AP_AHRS_DCM_ENABLED         case EKFTYPE::DCM:             return dcm.get_relative_position_NED_origin(vec);     #endif     #if HAL_NAVKF2_AVAILABLE         case EKFTYPE::TWO: {             Vector2p posNE; </pre>                    |
| 5     | AP_AHRS::get_velocity_NED()                 | returns state.velocity_NED                                        | Copter::crash_check; AR_PosControl                           | Data dependency         | <pre>bool AP_AHRS::get_velocity_NED(Vector3f &amp;vec) const {     vec = state.velocity_NED;     return state.velocity_NED_ok; }</pre>                                                                                                                                                                                                                          |
| 6     | NavEKF3_core::UpdateFilter()                | fill_scratch_variables(); Select*Fusion(); CovariancePrediction() | NavEKF3::UpdateFilter() (frontend, per core)                 | Function call           | <pre>void NavEKF3_core::UpdateFilter(bool predict) {     // don't run filter updates if states have not been initialised     if (!statesInitialised) {         return;     }      fill_scratch_variables();      // update sensor selection (for affinity) </pre>                                                                                               |

| Ref # | Component / Function               | Directly Calls                                                                    | Directly Called By                                | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------|------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7     | NavEKF3_core::SelectVelPosFusion() | readGpsData(); FuseVelPosNED()                                                    | NavEKF3_core::UpdateFilter()                      | Function call   | <pre> void NavEKF3_core::SelectVelPosFusion() {     // Check if the magnetometer has been fused on that time step     and the filter is running at faster than 200 Hz     // If so, don't fuse measurements on this time step to reduce     frame over-runs     // Only allow one time slip to prevent high rate magnetometer     data preventing fusion of other measurements     if (magFusePerformed &amp;&amp; dtIMUavg &lt; 0.005f &amp;&amp; !posVelFusionDelayed) {         posVelFusionDelayed = true;         return;     } else {         posVelFusionDelayed = false;     } } </pre> |
| 8     | NavEKF3_core::FuseVelPosNED()      | Kalman gain on NED pos/vel innovations; correct stateStruct                       | NavEKF3_core::SelectVelPosFusion()                | Function call   | <pre> void NavEKF3_core::FuseVelPosNED() {     // declare variables used to control access to arrays     bool fuseData[6] {};     uint8_t stateIndex;     uint8_t obsIndex;      // declare variables used by state and covariance update calculations     Vector6 R_OBS; // Measurement variances used for fusion } </pre>                                                                                                                                                                                                                                                                     |
| 9     | NavEKF3_core::readGpsData()        | gps.location(); gps.velocity();<br>gps.status() (via AP_DAL);<br>storedGPS.push() | NavEKF3_core::SelectVelPosFusion()/UpdateFilter() | Function call   | <pre> void NavEKF3_core::readGpsData() {     // check for new GPS data     const auto &amp;gps = dal.gps();      // limit update rate to avoid overflowing the FIFO buffer     if (gps.last_message_time_ms(selected_gps) - lastTimeGpsReceived_ms &lt;= frontend-&gt;sensorIntervalMin_ms) {         return;     } } </pre>                                                                                                                                                                                                                                                                    |
| 10    | NavEKF3_core::setAidingMode()      | checks gpsGoodToAlign, posTimeout;<br>sets PV_AidingMode                          | NavEKF3_core::controlFilterModes()                | Function call   | <pre> void NavEKF3_core::setAidingMode() {     resetDataSource posResetSource = resetDataSource::DEFAULT;     resetDataSource velResetSource = resetDataSource::DEFAULT;      // Save the previous status so we can detect when it has changed     PV_AidingModePrev = PV_AidingMode;      setYawSource(); } </pre>                                                                                                                                                                                                                                                                             |
| 11    | NavEKF3_core::SelectMagFusion()    | FuseMagnetometer(); FuseEulerYaw()                                                | NavEKF3_core::UpdateFilter()                      | Function call   | <pre> void NavEKF3_core::SelectMagFusion() {     // clear the flag that lets other processes know that the expensive magnetometer fusion operation has been performed on that time step     // used for load levelling     magFusePerformed = false;      // Store yaw angle when moving for use as a static reference when not moving     if (!onGroundNotMoving) {         if (fabsF(prevTnb[0][2]) &lt; fabsF(prevTnb[1][2])) { </pre>                                                                                                                                                       |

| Ref # | Component / Function               | Directly Calls                              | Directly Called By                          | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-------|------------------------------------|---------------------------------------------|---------------------------------------------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 12    | NavEKF3_core::SelectTasFusion()    | FuseAirspeed()                              | NavEKF3_core::UpdateFilter()                | Function call   | <pre> void NavEKF3_core::SelectTasFusion() {     // Check if the magnetometer has been fused on that time step     and the filter is running at faster than 200 Hz     // If so, don't fuse measurements on this time step to reduce     frame over-runs     // Only allow one time slip to prevent high rate magnetometer     data locking out fusion of other measurements     if (magFusePerformed &amp;&amp; dtIMUavg &lt; 0.005f &amp;&amp; !airSpdFusionDelayed) {         airSpdFusionDelayed = true;         return;     } else { </pre>   |
| 13    | NavEKF3_core::SelectFlowFusion()   | EstimateTerrainOffset(); FuseOptFlow()      | NavEKF3_core::UpdateFilter()                | Function call   | <pre> void NavEKF3_core::SelectFlowFusion() {     // Check if the magnetometer has been fused on that time step     and the filter is running at faster than 200 Hz     // If so, don't fuse measurements on this time step to reduce     frame over-runs     // Only allow one time slip to prevent high rate magnetometer     data preventing fusion of other measurements     if (magFusePerformed &amp;&amp; dtIMUavg &lt; 0.005f &amp;&amp; !optFlowFusionDelayed) {         optFlowFusionDelayed = true;         return;     } else { </pre> |
| 14    | NavEKF3_core::getLLH()             | outputDataNew / EKF_origin; builds Location | NavEKF3::getLLH() (frontend) -> AP_AHRS     | Function call   | <pre> bool NavEKF3_core::getLLH(Location &amp;loc) const {     Location origin;     if (getOriginLLH(origin)) {         postype_t posD;         if (getPosD_local(posD) &amp;&amp; PV_AidingMode != AID_NONE) {             // Altitude returned is an absolute altitude relative             to the WGS-84 spheroid             loc.set_alt_cm(origin.alt - posD*100.0, Location::AltFrame::ABSOLUTE);             if (filterStatus.flags.horiz_pos_abs    filterStatus.flags.horiz_pos_rel) { </pre>                                             |
| 15    | NavEKF3_core::calcGpsGoodToAlign() | reads gps metrics; sets gpsGoodToAlign      | NavEKF3_core::readGpsData()/setAidingMode() | Function call   | <pre> void NavEKF3_core::calcGpsGoodToAlign(void) {     if (inFlight &amp;&amp; !finalInflightYawInit &amp;&amp; assume_zero_sideslip() &amp;&amp; !use_compass()) {         // this is a special case where a plane has launched without magnetometer         // is now in the air and needs to align yaw to the GPS and start navigating as soon as possible         gpsGoodToAlign = true;         return;     } </pre>                                                                                                                         |
| 16    | NavEKF3_core::resetGyroBias()      | zeroes stateStruct.gyro_bias; resets P      | NavEKF3_core::InitialiseFilterBootstrap()   | Function call   | <pre> void NavEKF3_core::resetGyroBias(void) {     stateStruct.gyro_bias.zero();     zeroRows(P,10,12);     zeroCols(P,10,12);      P[10][10] = sq(radians(0.5f * dtIMUavg));     P[11][11] = P[10][10]; </pre>                                                                                                                                                                                                                                                                                                                                    |

| Ref # | Component / Function           | Directly Calls                                               | Directly Called By                        | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------|--------------------------------|--------------------------------------------------------------|-------------------------------------------|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 17    | NavEKF3_core::FuseRngBcn()     | range innovations; correct NED position states               | NavEKF3_core::SelectRngBcnFusion()        | Function call   | <pre> void NavEKF3_core::FuseRngBcn() {     // declarations     ftype pn;     ftype pe;     ftype pd;     ftype bcn_pn;     ftype bcn_pe;     ftype bcn_pd; </pre>                                                                                                                                                                                                                                                                                                                                                     |
| 18    | NavEKF3_core::Log_Write_XKF1() | getEulerAngles(); getVelNED(); getLLH(); AP::logger().Write* | NavEKF3_core::Log_Write()                 | Function call   | <pre> void NavEKF3_core::Log_Write_XKF1(uint64_t time_us) const {     // Write first EKF packet     Vector3f euler;     Vector2p posNE;     postype_t posD;     Vector3f velNED;     Vector3f gyroBias;     float posDownDeriv; </pre>                                                                                                                                                                                                                                                                                 |
| 19    | NavEKF3::getFilterStatus()     | core[].getFilterStatus()                                     | Copter::position_ok(); ekf_check()        | Function call   | <pre> void NavEKF3::getFilterStatus(nav_filter_status &amp;status) const {     if (core) {         core[primary].getFilterStatus(status);     } else {         memset(&amp;status, 0, sizeof(status));     } } </pre>                                                                                                                                                                                                                                                                                                  |
| 20    | NavEKF2_core::UpdateFilter()   | Select*Fusion(); CovariancePrediction()                      | NavEKF2::UpdateFilter() (frontend)        | Function call   | <pre> void NavEKF2_core::UpdateFilter(bool predict) {     // Set the flag to indicate to the filter that the front-end has     // given permission for a new state prediction cycle to be started     startPredictEnabled = predict;      // don't run filter updates if states have not been initialised     if (!statesInitialised) {         return;     } } </pre>                                                                                                                                                 |
| 21    | NavEKF2_core::FuseVelPosNED()  | Kalman gain on NED pos/vel innovations                       | NavEKF2_core::SelectVelPosFusion()        | Function call   | <pre> void NavEKF2_core::FuseVelPosNED() {     // health is set bad until test passed     bool velHealth = false; // boolean true if velocity measurements have passed innovation consistency check     bool posHealth = false; // boolean true if position measurements have passed innovation consistency check     bool hgtHealth = false; // boolean true if height measurements have passed innovation consistency check      // declare variables used to check measurement errors     Vector3F velInnov; </pre> |
| 22    | EKGFSF_yaw::update()           | predict()/correct() per model; AHRS complementary filter     | NavEKF3_core::runYawEstimatorPrediction() | Function call   | <pre> void EKGFSF_yaw::update(const Vector3F &amp;delAng,                         const Vector3F &amp;delVel,                         const ftype delAngDT,                         const ftype delVelDT,                         bool runEKF,                         ftype TAS) {     // copy to class variables </pre>                                                                                                                                                                                              |

| Ref # | Component / Function               | Directly Calls                                       | Directly Called By                         | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------|------------------------------------|------------------------------------------------------|--------------------------------------------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 23    | AC_WPNav::set_wp_destination_loc() | get_vector_NEU_cm();<br>set_wp_destination_NEU_cm()  | ArduCopter mode_auto / mode_guided         | Function call   | <pre> bool AC_WPNav::set_wp_destination_loc(const Location&amp; destination) {     bool is_terrain_alt;     Vector3f dest_neu_cm;      // convert destination location to vector     if (!get_vector_NEU_cm(destination, dest_neu_cm, is_terrain_alt)) {         return false;     } </pre>                                                                                                                                                                                                                                                                                                                                                                                            |
| 24    | AP_Mission::get_next_nav_cmd()     | get_next_cmd();<br>read_cmd_from_storage()           | ArduCopter/Plane mission sequencing        | Function call   | <pre> bool AP_Mission::get_next_nav_cmd(uint16_t start_index, Mission_Command&amp; cmd) {     // search until the end of the mission command list     for (uint16_t cmd_index = start_index; cmd_index &lt; (unsigned)_cmd_total; cmd_index++) {         // get next command         if (!get_next_cmd(cmd_index, cmd, false)) {             // no more commands so return failure             return false;         }         // if found a "navigation" command then return it     } </pre>                                                                                                                                                                                          |
| 25    | AP_L1_Control::update_waypoint()   | _ahrs.get_location(_current_loc)                     | ArduPlane navigation.cpp update loop       | Function call   | <pre> // Calculate L1 gain required for specified damping float K_L1 = 4.0f * _L1_damping * _L1_damping;  // Get current position and velocity if (_ahrs.get_location(_current_loc) == false) {     // if no GPS loc available, maintain last nav/target bearing     _data_is_stale = true;     return; } </pre>                                                                                                                                                                                                                                                                                                                                                                       |
| 26    | AP_L1_Control::nav_roll_cd()       | _ahrs.get_pitch_rad(); _latAccDem                    | Plane::calc_nav_roll() (Attitude.cpp L608) | Function call   | <pre> int32_t AP_L1_Control::nav_roll_cd(void) const {     float ret;     /*      * formula can be obtained through equations of balanced spiral:      * liftForce * cos(roll) = gravityForce * cos(pitch);      * liftForce * sin(roll) = gravityForce * lateralAcceleration / gravityAcceleration; // as mass = gravityForce/gravityAcceleration      * see issue 24319 [https://github.com/ArduPilot/ardupilot/issues/24319]      * Multiplier 100.0f is for converting degrees to centidegrees      * Made changes to avoid zero division as proposed by Andrew Tridgell: https://github.com/ArduPilot/ardupilot/pull/24331#discussion_r1267798397      */     ret = ...; } </pre> |
| 27    | AP_TECS::update_pitch_throttle()   | _update_pitch();<br>_update_throttle_with_airspeed() | ArduPlane stabilize/auto pitch loop        | Function call   | <pre> void AP_TECS::update_pitch_throttle(int32_t hgt_dem_cm,                                    int32_t EAS_dem_cm,                                    enum AP_FixedWing::FlightStage flight_stage,                                    float distance_beyond_land_wp,                                    int32_t ptchMinCO_cd,                                    int16_t throttle_nudge,                                    float hgt_ave,                                    float load_factor,                                    float pitch_trim_deg) { </pre>                                                                                                                   |

| Ref # | Component / Function                                      | Directly Calls                         | Directly Called By                     | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------|-----------------------------------------------------------|----------------------------------------|----------------------------------------|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 28    | AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd() | input_euler_angle_roll_pitch_yaw_rad() | ArduCopter/ArduPlane flight-mode run() | Function call   | <pre> void AC_AttitudeControl::input_euler_angle_roll_pitch_yaw_cd(float euler_roll_angle_cd, float euler_pitch_angle_cd, float euler_yaw_ angle_cd, bool slew_yaw) {     // Convert from centidegrees on public interface to radians     const float euler_roll_angle_rad = cd_to_rad(euler_roll_angle_ cd);     const float euler_pitch_angle_rad = cd_to_rad(euler_pitch_angl e_cd);     const float euler_yaw_angle_rad = cd_to_rad(euler_yaw_angle_cd );      input_euler_angle_roll_pitch_yaw_rad(euler_roll_angle_rad, eul er_pitch_angle_rad, euler_yaw_angle_rad, slew_yaw); } </pre> |

**Layer 2 — Full transitive chain to final consumer (hop count)**

| Ref # | PNT Origin                                  | Full Dependency Chain                                                                                                                   | Final Consumer                                     | Chain Depth | Notes                                                                                                                                                                                                                                                                    |
|-------|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | AP_GPS (via AP_AHRS) — user example         | AP_GPS → AP_AHRS::get_location() → Plane::navigate() [ArduPlane/navigation.cpp:86] → ModeAuto::navigate() [ArduPlane/mode_auto.cpp:124] | ModeAuto::navigate() [ArduPlane/mode_auto.cpp:124] | 3           | USER-SUPPLIED example (AAP 0.1.3/0.5.4). The literal example named 'ArduPlane::flight_mode_auto()', which does NOT exist in this snapshot; normalized to the real symbols Plane::navigate() and ModeAuto::navigate(). navigate() is gated on have_position (Table 5 #5). |
| 2     | AP_AHRS::groundspeed()                      | AP_GPS velocity → NavEKF3 fusion → AP_AHRS::groundspeed() → Rover speed control → SRV_Channels                                          | SRV_Channels                                       | 4           | Ground-vehicle speed branch.                                                                                                                                                                                                                                             |
| 3     | AP_AHRS::have_inertial_nav()                | EKF health → AP_AHRS::have_inertial_nav() → Copter::ekf_has_absolute_position → position_ok() → set_mode gate                           | mode transition (allow/deny)                       | 4           | Also observed in Group 2 (Table 5 #7).                                                                                                                                                                                                                                   |
| 4     | AP_AHRS::get_relative_position_NED_origin() | EKF → AHRS NED dispatch → {AR_PosControl   AP_InertialNav} → actuators                                                                  | SRV_Channels / AP_Motors                           | 3           | Geodetic↔NED transform branch.                                                                                                                                                                                                                                           |
| 5     | AP_AHRS::get_velocity_NED()                 | EKF → AHRS → Copter::crash_check / controllers → disarm / AP_Motors                                                                     | AP_Motors / disarm                                 | 3           | Velocity consumers.                                                                                                                                                                                                                                                      |
| 6     | NavEKF3_core::UpdateFilter()                | {AP_GPS, AP_InertialSensor, AP_Baro} → NavEKF3::UpdateFilter → AP_AHRS estimate → all consumers                                         | AP_Motors                                          | 3           | [CIRCULAR] AHRS↔EKF; EKF reads AP_DAL.                                                                                                                                                                                                                                   |
| 7     | NavEKF3 PosVel fusion (Select)              | readGpsData buffer → SelectVelPosFusion → FuseVelPosNED → EKF states → AP_AHRS                                                          | AP_AHRS                                            | 4           | Core fusion scheduling.                                                                                                                                                                                                                                                  |
| 8     | NavEKF3_core::FuseVelPosNED()               | GPS NED innovations → FuseVelPosNED → stateStruct correction → getLLH → AP_AHRS::get_location()                                         | AP_AHRS                                            | 4           | Position/velocity measurement-update.                                                                                                                                                                                                                                    |
| 9     | NavEKF3_core::readGpsData()                 | AP_GPS::location()/velocity() → readGpsData → storedGPS buffer → SelectVelPosFusion → FuseVelPosNED                                     | EKF state correction                               | 4           | Explicit GPS→EKF ingestion hop (cf. Table 1a chains).                                                                                                                                                                                                                    |
| 10    | NavEKF3_core::setAidingMode()               | GPS health → setAidingMode → PV_AidingMode → SelectVelPosFusion → EKF states                                                            | EKF aiding state                                   | 4           | Nav aiding-mode control.                                                                                                                                                                                                                                                 |
| 11    | NavEKF3_core::SelectMagFusion()             | magnetometer → SelectMagFusion → FuseMagnetometer → yaw state → AP_AHRS heading                                                         | AP_AHRS                                            | 4           | Heading component.                                                                                                                                                                                                                                                       |
| 12    | NavEKF3_core::SelectTasFusion()             | airspeed → SelectTasFusion → FuseAirspeed → wind/vel states → AP_AHRS::wind_estimate                                                    | AP_AHRS                                            | 4           | Wind/velocity.                                                                                                                                                                                                                                                           |
| 13    | NavEKF3_core::SelectFlowFusion()            | optical flow → SelectFlowFusion → FuseOptFlow → vel/pos states → AP_AHRS                                                                | AP_AHRS                                            | 4           | GPS-denied aiding.                                                                                                                                                                                                                                                       |
| 14    | NavEKF3_core::getLLH()                      | EKF state → getLLH → NavEKF3::getLLH (frontend) → AP_AHRS::get_location() → nav consumers                                               | AP_AHRS / consumers                                | 4           | EKF position output edge.                                                                                                                                                                                                                                                |
| 15    | NavEKF3_core::calcGpsGoodToAlign()          | AP_GPS metrics → calcGpsGoodToAlign → gpsGoodToAlign → setAidingMode → EKF origin set                                                   | EKF origin / aiding                                | 4           | GPS quality gate.                                                                                                                                                                                                                                                        |
| 16    | NavEKF3_core::resetGyroBias()               | InitialiseFilterBootstrap → resetGyroBias → stateStruct.gyro_bias → state prediction                                                    | EKF prediction                                     | 3           | Bias init.                                                                                                                                                                                                                                                               |
| 17    | NavEKF3_core::FuseRngBcn()                  | AP_Beacon ranges → FuseRngBcn → EKF NED position correction → AP_AHRS                                                                   | AP_AHRS                                            | 3           | Non-GPS position fusion.                                                                                                                                                                                                                                                 |

| Ref # | PNT Origin                       | Full Dependency Chain                                                                         | Final Consumer       | Chain Depth | Notes                                        |
|-------|----------------------------------|-----------------------------------------------------------------------------------------------|----------------------|-------------|----------------------------------------------|
| 18    | NavEKF3_core::Log_Write_XKF1()   | EKF states → Log_Write_XKF1 → AP_Logger → dataflash/SD log                                    | dataflash log        | 3           | Logging Sink branch.                         |
| 19    | NavEKF3::getFilterStatus()       | EKF core status bits → NavEKF3::getFilterStatus → AP_AHRS → Copter::position_ok()/ekf_check() | vehicle health logic | 3           | Status output feeds Group 2.                 |
| 20    | NavEKF2_core::UpdateFilter()     | {AP_GPS, INS, Baro} → NavEKF2::UpdateFilter → AP_AHRS (when EKF2 active) → consumers          | AP_Motors            | 3           | [CIRCULAR][DUPLICATED] parallel EKF to EKF3. |
| 21    | NavEKF2_core::FuseVelPosNED()    | GPS NED → NavEKF2 FuseVelPosNED → EKF2 states → AP_AHRS                                       | AP_AHRS              | 3           | [DUPLICATED] mirrors EKF3 #8.                |
| 22    | EKFGSF_yaw::update()             | AP_GPS velocity + IMU → EKFGSF_yaw::update → emergency yaw → EKF yaw reset → AP_AHRS          | AP_AHRS              | 4           | Yaw-recovery branch.                         |
| 23    | AC_WPNav                         | AP_AHRS pos → AC_WPNav target → AC_PosControl → AC_AttitudeControl → AP_Motors                | AP_Motors            | 4           | Multirotor waypoint path.                    |
| 24    | AP_Mission                       | mission storage → get_next_nav_cmd → mode_auto → AC_WPNav → controllers                       | AP_Motors            | 4           | Mission sequencing path.                     |
| 25    | AP_L1_Control::update_waypoint() | AP_AHRS pos → L1 guidance → nav_roll_cd() → Plane::calc_nav_roll → SRV_Channels               | SRV_Channels         | 4           | Fixed-wing lateral path.                     |
| 26    | AP_L1_Control::nav_roll_cd()     | L1 latAccDem → nav_roll_cd() → Plane::calc_nav_roll → SRV_Channels                            | SRV_Channels         | 3           | Roll-demand branch.                          |
| 27    | AP_TECS                          | airspeed/alt demand → TECS update_pitch_throttle → pitch/throttle → SRV_Channels              | SRV_Channels         | 3           | Energy/altitude branch.                      |
| 28    | AC_AttitudeControl (Sink)        | nav demand → input_euler_* → attitude_controller_run_quat → rate_controller_run → AP_Motors   | AP_Motors            | 4           | Inbound chain to attitude Sink.              |



Table 3 — Core Timing (HAL clock primitives, RTC &amp; time-source arbitration, jitter correction, scheduler, GPS time-of-week)

| # | File Path                                   | Line(s) | Function / Class                                            | Role      | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                              | Notes                                                                                                                                                                                 |
|---|---------------------------------------------|---------|-------------------------------------------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | libraries/AP_HAL/syst<br>em.h               | 15-20   | AP_HAL::micros() /<br>millis() / micros64() /<br>millis64() | Source    | uint16_t micros16();<br>uint32_t micros();<br>uint32_t millis();<br>uint16_t millis16();<br>uint64_t micros64();<br>uint64_t millis64();                                                                                                                                                                                                                                                                                               | Monotonic time backbone (user-cited micros()/millis()). Free functions in the AP_HAL namespace backed by the board RTOS timer; the time primitive every PNT subsystem stamps against. |
| 2 | libraries/AP_RTC/AP<br>_RTC.h               | 32-40   | AP_RTC::get_utc_usec<br>()                                  | Source    | /*<br>get clock in UTC microseconds. Returns false if it is not a<br>available.<br>*/<br>bool get_utc_usec(uint64_t &usec) const;<br><br>// set the system time. If the time has already been set by<br>// something better (according to source_type), this set will<br>be<br>// ignored.<br>void set_utc_usec(uint64_t time_utc_usec, source_type type);                                                                             | UTC wall-clock time in microseconds (returns false if unavailable). The absolute-time API consumed for logging and GPS-time conversion.                                               |
| 3 | libraries/AP_RTC/AP<br>_RTC.cpp             | 56-65   | AP_RTC::set_utc_usec<br>()                                  | Transform | // only allow time to be moved forward from the same sourcetype<br><br>// while the vehicle is disarmed:<br>if (hal.util->get_soft_armed()) {<br>if (type >= rtc_source_type) {<br>// e.g. system-time message when we've been set by the<br>GPS<br>return;<br>}<br>} else {<br>// vehicle is disarmed; accept (e.g.) GPS time source updates<br>if (type > rtc_source_type) {                                                         | Time-SOURCE ARBITRATION: accepts a new UTC time only if its source_type out-ranks the current rtc_source_type (GPS > GCS > HW). Selects the best clock source.                        |
| 4 | libraries/AP_RTC/Jitt<br>erCorrection.cpp   | 50-58   | JitterCorrection::correct<br>_offboard_timestamp_u<br>sec() | Transform | uint64_t JitterCorrection::correct_offboard_timestamp_usec(uint64_t offboard_usec, uint64_t local_usec)<br>{<br>int64_t diff_us = int64_t(local_usec) - int64_t(offboard_usec)<br>;<br><br>if (!initialised   <br>diff_us < link_offset_usec) {<br>// this message arrived from the remote system with a<br>// timestamp that would imply the message was from the<br>// future. We know that isn't possible, so we adjust down<br>the | Time synchronization: maps an offboard (remote-system) timestamp onto the local clock by tracking the minimum observed link offset — rejects 'from the future' samples.               |
| 5 | libraries/AP_Schedul<br>er/AP_Scheduler.cpp | 354-363 | AP_Scheduler::loop()                                        | Transform | hal.util->persistent_data.scheduler_task = -1;<br><br>_loop_sample_time_us = AP_HAL::micros64();<br>const uint32_t sample_time_us = uint32_t(_loop_sample_time_us)<br>;<br><br>if (_loop_timer_start_us == 0) {<br>_loop_timer_start_us = sample_time_us;<br>_last_loop_time_s = get_loop_period_s();<br>} else {<br>_last_loop_time_s = (sample_time_us - _loop_timer_start_us<br>) * 1.0e-6;                                         | Main-loop timing core: stamps each loop with AP_HAL::micros64() and derives the measured loop time. Drives the cadence of every periodic PNT task (EKF predict/fuse, AHRS, control).  |
| 6 | libraries/AP_Schedul<br>er/AP_Scheduler.cpp | 168-176 | AP_Scheduler::tick()                                        | Source    | void AP_Scheduler::tick(void)<br>{<br>_tick_counter++;<br>_tick_counter32++;<br>}<br><br>#if CONFIG_HAL_BOARD == HAL_BOARD_SITL<br>/*<br>fill stack with NaN so we can catch use of uninitialised stack                                                                                                                                                                                                                                | Increments the loop tick counters (_tick_counter / _tick_counter32) that index task scheduling intervals.                                                                             |

| # | File Path                             | Line(s) | Function / Class                     | Role      | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                        | Notes                                                                                                                                                                                                 |
|---|---------------------------------------|---------|--------------------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7 | libraries/AP_Scheduler/AP_Scheduler.h | 149-155 | AP_Scheduler::get_loop_period_us()   | Transform | <pre>// get the time-allowed-per-loop in microseconds uint32_t get_loop_period_us() {     if (_loop_period_us == 0) {         _loop_period_us = 1000000UL / _loop_rate_hz;     }     return _loop_period_us; }</pre>                                                             | Loop-rate→period derivation: <code>_loop_period_us = 1000000UL / _loop_rate_hz</code> . Couples the configured loop rate to the per-loop time budget and to controller/EKF dt.                        |
| 8 | libraries/AP_GPS/AP_GPS.h             | 409-418 | AP_GPS::time_week_ms() / time_week() | Source    | <pre>// GPS time of week in milliseconds uint32_t time_week_ms(uint8_t instance) const {     return state[instance].time_week_ms; } uint32_t time_week_ms() const {     return time_week_ms(primary_instance); }  // GPS week uint16_t time_week(uint8_t instance) const {</pre> | [MULTI-CATEGORY] GPS time-of-week (ms-of-week + week number) — the TIMING facet of the GPS_State that also carries Positioning (Table 1 #4/#17). Emitted separately here per the multi-category rule. |

Table 3a — Timing dependencies (Layer 1 direct edges + Layer 2 transitive chains)

**Layer 1 — Direct calls / called-by with typed dependency**

| Ref # | Component / Function                                | Directly Calls                                                      | Directly Called By                                                       | Dependency Type           | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------|-----------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | AP_HAL::micros() / millis()                         | board RTOS/HAL timer backend                                        | EKF, AP_Scheduler, AP_GPS, all failsafes (ubiquitous)                    | Function call             | <pre>uint16_t micros16(); uint32_t micros(); uint32_t millis(); uint16_t millis16(); uint64_t micros64(); uint64_t millis64();</pre>                                                                                                                                                                                                                                                                                                                            |
| 2     | AP_RTC::get_utc_usec() — via AP::rtc()              | AP::rtc().get_utc_usec(rtc)                                         | AP_Scheduler::Log_Write_Performance; AP_Logger; AP_Stats; AP_NMEA_Output | Shared global / singleton | <pre>void AP_Scheduler::Log_Write_Performance() {     uint64_t rtc = 0;     #if AP_RTC_ENABLED         UNUSED_RESULT(AP::rtc().get_utc_usec(rtc));     #endif      const AP_HAL::Util::PersistentData &amp;pd = hal.util-&gt;persistent_data;     struct log_Performance pkt = {</pre>                                                                                                                                                                          |
| 3     | AP_RTC::set_utc_usec() (source arbitration)         | reads hal.util->get_soft_armed(); compares & writes rtc_source_type | AP_GPS (GPS time); GCS SYSTEM_TIME; AP_BoardConfig (HW RTC)              | Data dependency           | <pre>// only allow time to be moved forward from the same sourcetype // while the vehicle is disarmed: if (hal.util-&gt;get_soft_armed()) {     if (type &gt;= rtc_source_type) {         // e.g. system-time message when we've been set by the GPS         return;     } } else {     // vehicle is disarmed; accept (e.g.) GPS time source updates     if (type &gt; rtc_source_type) {</pre>                                                                |
| 4     | JitterCorrection::correct_offboard_timestamp_usec() | computes diff_us; updates link_offset_usec                          | AP_RTC; DroneCAN/MAVLink timestamp consumers                             | Function call             | <pre>uint64_t JitterCorrection::correct_offboard_timestamp_usec(uint64_t offboard_usec, uint64_t local_usec) {     int64_t diff_us = int64_t(local_usec) - int64_t(offboard_usec); ;      if (!initialised            diff_us &lt; link_offset_usec) {         // this message arrived from the remote system with a         // timestamp that would imply the message was from the         // future. We know that isn't possible, so we adjust down the</pre> |

| Ref # | Component / Function                    | Directly Calls                                                            | Directly Called By                                                | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                                                                              |
|-------|-----------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5     | AP_Scheduler::loop()                    | AP_HAL::micros64();<br>get_loop_period_s(); dispatches<br>run-table tasks | AP_Vehicle::loop() via scheduler.loop()<br>(AP_Vehicle.cpp:545)   | Function call   | <pre> hal.util-&gt;persistent_data.scheduler_task = -1;  _loop_sample_time_us = AP_HAL::micros64(); const uint32_t sample_time_us = uint32_t(_loop_sample_time_us) ;  if (_loop_timer_start_us == 0) {     _loop_timer_start_us = sample_time_us;     _last_loop_time_s = get_loop_period_s(); } else {     _last_loop_time_s = (sample_time_us - _loop_timer_start_us ) * 1.0e-6; </pre> |
| 6     | AP_Scheduler::tick()                    | increments _tick_counter /<br>_tick_counter32                             | AP_Scheduler::loop(); fast-loop                                   | Function call   | <pre> void AP_Scheduler::tick(void) {     _tick_counter++;     _tick_counter32++; }  #ifdef CONFIG_HAL_BOARD == HAL_BOARD_SITL /* fill stack with NaN so we can catch use of uninitialised stack </pre>                                                                                                                                                                                   |
| 7     | AP_Scheduler::get_loop_period_us()      | derives _loop_period_us from<br>_loop_rate_hz                             | task budgeting; controller dt (G_Dt)                              | Data dependency | <pre> // get the time-allowed-per-loop in microseconds uint32_t get_loop_period_us() {     if (_loop_period_us == 0) {         _loop_period_us = 1000000UL / _loop_rate_hz;     }     return _loop_period_us; } </pre>                                                                                                                                                                    |
| 8     | AP_GPS::time_week_ms() /<br>time_week() | returns state[instance].time_week_ms /<br>time_week                       | NavEKF3 readGpsData (GPS time);<br>time_epoch conversion; logging | Data dependency | <pre> // GPS time of week in milliseconds uint32_t time_week_ms(uint8_t instance) const {     return state[instance].time_week_ms; }  uint32_t time_week_ms() const {     return time_week_ms(primary_instance); }  // GPS week uint16_t time_week(uint8_t instance) const { </pre>                                                                                                       |

**Layer 2 — Full transitive chain to final consumer (hop count)**

| Ref # | PNT Origin                       | Full Dependency Chain                                                                                         | Final Consumer                | Chain Depth | Notes                                   |
|-------|----------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------------------|-------------|-----------------------------------------|
| 1     | RTOS/board hardware timer        | board timer → AP_HAL::micros()/millis() → AP_Scheduler / EKF / AP_GPS timestamps → every<br>periodic loop     | all periodic PNT subsystems   | 3           | Monotonic-time backbone.                |
| 2     | GPS/GCS UTC time source          | time source → AP_RTC → AP::rtc().get_utc_usec() → AP_Logger / AP_Stats → log timestamps                       | dataflash log timestamps      | 4           | Singleton consumption edge (AP::rtc()). |
| 3     | AP_GPS time / GCS<br>SYSTEM_TIME | time source → set_utc_usec source arbitration → rtc_source_type → system UTC clock                            | RTC system clock              | 3           | Best-source selection.                  |
| 4     | Offboard (remote) timestamp      | MAVLink/DroneCAN timestamp → correct_offboard_timestamp_usec → corrected local time → EKF<br>/ sensor fusion  | fused-sensor timing           | 3           | Time-sync transform.                    |
| 5     | AP_HAL::micros64() (loop clock)  | AP_HAL::micros64() → AP_Scheduler::loop() → run-table tasks (EKF/AHRS/controllers) →<br>AP_Motors             | AP_Motors (all tasks)         | 3           | Loop drives every PNT task.             |
| 6     | AP_Scheduler::loop()             | loop() → tick() counters → task-table interval check → periodic task dispatch                                 | periodic task dispatch        | 3           | Cadence counter.                        |
| 7     | SCHED_LOOP_RATE parameter        | _loop_rate_hz → get_loop_period_us (1e6/rate) → task budget + G_Dt → AC_AttitudeControl / EKF<br>dt           | controller / EKF dt           | 3           | Rate→period coupling.                   |
| 8     | GPS receiver (time-of-week)      | GPS receiver → GPS_State.time_week → AP_GPS::time_week() → NavEKF3 readGpsData /<br>time_epoch_usec → UTC/log | EKF GPS-time / UTC conversion | 4           | [MULTI-CATEGORY] timing facet of GPS.   |

## GROUP 2 — INDIRECT / RELATIONAL PNT REFERENCES

INDIRECT / RELATIONAL — not core. This code does not directly read or write PNT data; its behavior changes as a result of PNT state. Each row records Trigger Condition, PNT State Observed (the discriminator from Group 1), and Behavior Change Description.

**Table 4 — Indirect / Relational — Positioning (mode position-gating, geofence, GPS-denied fallback)**

| # | File Path                       | Line(s) | Function / Class                       | Trigger Condition                                                          | PNT State Observed                                                                                               | Behavior Change Description                                                                                                     | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Notes                                                                                                                                                                                                                                      |
|---|---------------------------------|---------|----------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | ArduCopter/system.cpp           | 226-234 | Copter::position_ok()                  | Evaluated on position-mode entry and by position-dependent failsafes       | failsafe.ekf flag; ekf_has_absolute_position() / ekf_has_relative_position() (EKF position validity)             | Returns false when EKF failsafe is set or no EKF position estimate exists, causing callers to deny position-requiring behavior. | <pre>bool Copter::position_ok() const {     // return false if ekf failsafe has triggered     if (failsafe.ekf) {         return false;     }      // check ekf position estimate     return (ekf_has_absolute_position()    ekf_has_relative_position()); }</pre>                                                                                                                                                                                                                         | INDIRECT/RELATIONAL (positioning). Pure observer of EKF position health — reads no raw PNT, but gates downstream behavior.                                                                                                                 |
| 2 | ArduCopter/mode.cpp             | 303-310 | Copter::set_mode() — requires_GPS gate | Pilot/GCS/auto requests a flight-mode change                               | new_flightmode->requires_GPS() && copter.position_ok()                                                           | Rejects the mode change with "requires position" when the target mode needs GPS and position_ok() is false.                     | <pre>if (!ignore_checks &amp;&amp;     new_flightmode-&gt;requires_GPS() &amp;&amp;     !copter.position_ok()) {     mode_change_failed(new_flightmode, "requires position");     return false; }</pre>                                                                                                                                                                                                                                                                                    | INDIRECT/RELATIONAL (positioning). Central position-gate for ALL position-requiring Copter modes (per-mode requires_GPS() enumerated in the coverage register).                                                                            |
| 3 | ArduCopter/fence.cpp            | 20-28   | Copter::fence_checks_async()           | 25 Hz fence rate-gate elapsed and no breach latched awaiting the main loop | fence.get_breaches() (current breach bitmask, L21); fence.check() (re-evaluates vehicle position vs fences, L25) | Latches new_breaches bitmask for the main loop and logs a breach-cleared event when the vehicle returns inside the fence.       | <pre>fence_breaches.last_check_ms = now; const uint8_t orig_breaches = fence.get_breaches(); bool is_landing_or_landed = flightmode-&gt;is_landing()    ap.land_complete    !motors-&gt;armed();  // check for new breaches; new_breaches is bitmask of fence types breached fence_breaches.new_breaches = fence.check(is_landing_or_landed);  if (!fence_breaches.new_breaches &amp;&amp; orig_breaches &amp;&amp; fence.get_breaches() == 0) {     if (!copter.ap.land_complete) {</pre> | [FIX F1] Cited window centers on get_breaches() (L21) and check() (L25) — the actual geofence observation — not the L8-17 async/timing entry. [DUPLICATED] mirrors Rover::fence_checks_async (#9). Timing rate-gate documented in Table 6. |
| 4 | libraries/AC_Fence/AC_Fence.cpp | 677-685 | AC_Fence::check_fence_polygon()        | Polygon (inclusion/exclusion) fence enabled                                | AP::ahrs().get_location(loc) — current vehicle Location tested against the polygon                               | record_breach(POLYGON) when _poly_loader.breached(loc) is true → sets the polygon breach bit.                                   | <pre>const bool was_breached = _breached_fences &amp; AC_FENCE_TYPE_POLYGON;  Location loc; const bool have_location = AP::ahrs().get_location(loc);  if (have_location &amp;&amp; _poly_loader.breached(loc, _polygon_breach_distance)) {     if (!was_breached) {         record_breach(AC_FENCE_TYPE_POLYGON);         return true;     } }</pre>                                                                                                                                       | [FIX F2] Real position-observing path (was mis-cited to L753-762 mask init). Snippet centers on the AP::ahrs().get_location() read (L680).                                                                                                 |

| # | File Path                        | Line(s) | Function / Class                | Trigger Condition                                                  | PNT State Observed                                                                                           | Behavior Change Description                                                                                 | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                               | Notes                                                                                                                                                   |
|---|----------------------------------|---------|---------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 | libraries/AC_Fence/AC_Fence.cpp  | 707-715 | AC_Fence::check_fence_circle()  | Circle fence enabled                                               | AP::ahrs().get_relative_position_NE_home(home) — NE distance from home (L713)                                | Records a circle breach when the home distance exceeds the configured radius.                               | <pre> if (!(get_enabled_fences() &amp; AC_FENCE_TYPE_CIRCLE)) {     // not enabled; no breach     return false; }  Vector2f home; if (AP::ahrs().get_relative_position_NE_home(home)) {     // we (may) remain breached if we can't update home     _home_distance = home.length(); </pre>                                                                                                                                                              | [FIX F2] Real position-observing path; snippet centers on the NE-home position read.                                                                    |
| 6 | libraries/AC_Fence/AC_Fence.cpp  | 540-548 | AC_Fence::check_fence_alt_max() | Maximum-altitude fence enabled                                     | AP::ahrs().get_relative_position_D_home(alt) — altitude relative to home (L546)                              | Records an altitude breach when alt exceeds _alt_max.                                                       | <pre> if (!(get_enabled_fences() &amp; AC_FENCE_TYPE_ALT_MAX)) {     // not enabled; no breach     return false; }  float alt; AP::ahrs().get_relative_position_D_home(alt); const float _curr_alt = -alt; // translate Down to Up </pre>                                                                                                                                                                                                               | [FIX F2] Real altitude-observing path; snippet centers on the D-home altitude read.                                                                     |
| 7 | libraries/AC_Fence/AC_Fence.cpp  | 837-845 | AC_Fence::check()               | Periodic fence check invoked from the vehicle fence_checks_async() | (aggregator) results of check_fence_alt_max() / check_fence_circle() / check_fence_polygon() position checks | Returns the bitmask of breached fence types that drives the vehicle fence failsafe action.                  | <pre> if (!(disabled_fences &amp; AC_FENCE_TYPE_CIRCLE) &amp;&amp; check_fence_circle()) {     ret  = AC_FENCE_TYPE_CIRCLE; }  // polygon fence check if (!(disabled_fences &amp; AC_FENCE_TYPE_POLYGON) &amp;&amp; check_fence_polygon()) {     ret  = AC_FENCE_TYPE_POLYGON; } </pre>                                                                                                                                                                 | [FIX F2] Orchestrator dispatch (L837 circle, L842 polygon). The position reads themselves are in the dispatched checks (#4-#6).                         |
| 8 | ArduCopter/mode_guided_nogps.cpp | 9-15    | ModeGuidedNoGPS::init()         | GUIDED_NOGPS entry (companion-computer angle control without GPS)  | [ABSENT — by design] no position state observed; requires_GPS() returns false (ArduCopter/mode.h)            | Starts angle control only and remains available when position_ok() is false — the GPS-denied fallback path. | <pre> // initialise guided_nogps controller bool ModeGuidedNoGPS::init(bool ignore_checks) {     // start in angle control mode     ModeGuided::angle_control_start();     return true; } </pre>                                                                                                                                                                                                                                                        | INDIRECT/RELATIONAL (positioning). Contrast row: intentionally PNT-position-INDEPENDENT mode (documents an explicit absence of positioning dependency). |
| 9 | Rover/fence.cpp                  | 17-25   | Rover::fence_checks_async()     | Rover fence rate-gate elapsed and no breach latched                | fence.get_breaches() (L19); fence.check() (vehicle position vs fence, L21)                                   | Latches new_breaches for the main loop; clears on return inside the fence.                                  | <pre> fence_breaches.last_check_ms = now; const uint8_t orig_breaches = fence.get_breaches(); // check for new breaches; new_breaches is bitmask of fence types breached fence_breaches.new_breaches = fence.check();  if (!fence_breaches.new_breaches &amp;&amp; orig_breaches &amp;&amp; fence.get_breaches() == 0) {     // record clearing of breach     LOGGER_WRITE_ERROR(LogErrorSubsystem::FAILSAFE_FENCE, LogError::ERROR_RESOLVED); } </pre> | [DUPLICATED] structurally identical to ArduCopter::fence_checks_async (#3) — copy-paste across vehicles (extraction risk).                              |

Table 4a — Indirect Positioning dependencies (Layer 1 + Layer 2)

Layer 1 — Direct calls / called-by with typed dependency

| Ref # | Component / Function                    | Directly Calls                                                                     | Directly Called By                                               | Dependency Type              | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------|-----------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Copter::position_ok()                   | ekf_has_absolute_position();<br>ekf_has_relative_position(); reads<br>failsafe.ekf | Copter::set_mode (mode.cpp:306);<br>position-dependent failsafes | Function call                | <pre> bool Copter::position_ok() const {     // return false if ekf failsafe has triggered     if (failsafe.ekf) {         return false;     }      // check ekf position estimate     return (ekf_has_absolute_position()    ekf_has_relative_positi on()); </pre>                                                                                                                                                                                                                              |
| 2     | Copter::set_mode() requires_GPS<br>gate | new_flightmode->requires_GPS();<br>copter.position_ok()                            | GCS/RC/auto mode-change entry points                             | Function call                | <pre> if (!ignore_checks &amp;&amp;     new_flightmode-&gt;requires_GPS() &amp;&amp;     !copter.position_ok()) {     mode_change_failed(new_flightmode, "requires position");     return false; } </pre>                                                                                                                                                                                                                                                                                        |
| 3     | Copter::fence_checks_async()            | fence.get_breaches(); fence.check()                                                | Copter IO thread (1 kHz async, 25 Hz<br>gated)                   | Function call                | <pre> fence_breaches.last_check_ms = now; const uint8_t orig_breaches = fence.get_breaches(); bool is_landing_or_landed = flightmode-&gt;is_landing()    ap.lan d_complete    !motors-&gt;armed();  // check for new breaches; new_breaches is bitmask of fence ty pes breached fence_breaches.new_breaches = fence.check(is_landing_or_landed );  if (!fence_breaches.new_breaches &amp;&amp; orig_breaches &amp;&amp; fence.get _breaches() == 0) {     if (!copter.ap.land_complete) { </pre> |
| 4     | AC_Fence::check_fence_polygon()         | AP::ahrs().get_location();<br>_poly_loader.breached()                              | AC_Fence::check() (L842)                                         | Shared global /<br>singleton | <pre> const bool was_breached = _breached_fences &amp; AC_FENCE_TYPE_POL YGON;  Location loc; const bool have_location = AP::ahrs().get_location(loc);  if (have_location &amp;&amp; _poly_loader.breached(loc, _polygon_breac h_distance)) {     if (!was_breached) {         record_breach(AC_FENCE_TYPE_POLYGON);         return true; </pre>                                                                                                                                                 |
| 5     | AC_Fence::check_fence_circle()          | AP::ahrs().get_relative_position_NE_ho<br>me()                                     | AC_Fence::check() (L837)                                         | Shared global /<br>singleton | <pre> if (!(get_enabled_fences() &amp; AC_FENCE_TYPE_CIRCLE)) {     // not enabled; no breach     return false; }  Vector2f home; if (AP::ahrs().get_relative_position_NE_home(home)) {     // we (may) remain breached if we can't update home     _home_distance = home.length(); </pre>                                                                                                                                                                                                       |
| 6     | AC_Fence::check_fence_alt_max()         | AP::ahrs().get_relative_position_D_ho<br>me()                                      | AC_Fence::check() (L821)                                         | Shared global /<br>singleton | <pre> if (!(get_enabled_fences() &amp; AC_FENCE_TYPE_ALT_MAX)) {     // not enabled; no breach     return false; }  float alt; AP::ahrs().get_relative_position_D_home(alt); const float _curr_alt = -alt; // translate Down to Up </pre>                                                                                                                                                                                                                                                        |

| Ref # | Component / Function        | Directly Calls                                                           | Directly Called By                                           | Dependency Type         | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------|-----------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7     | AC_Fence::check()           | check_fence_alt_max();<br>check_fence_circle();<br>check_fence_polygon() | Copter::fence_checks_async();<br>Rover::fence_checks_async() | Function call           | <pre> if (!(disabled_fences &amp; AC_FENCE_TYPE_CIRCLE) &amp;&amp; check_fence_circle()) {     ret  = AC_FENCE_TYPE_CIRCLE; }  // polygon fence check if (!(disabled_fences &amp; AC_FENCE_TYPE_POLYGON) &amp;&amp; check_fence_polygon()) {     ret  = AC_FENCE_TYPE_POLYGON; } </pre>                                                                                                                                                                 |
| 8     | ModeGuidedNoGPS::init()     | ModeGuided::angle_control_start()                                        | Copter::set_mode()                                           | Inheritance / Interface | <pre> // initialise guided_nogps controller bool ModeGuidedNoGPS::init(bool ignore_checks) {     // start in angle control mode     ModeGuided::angle_control_start();     return true; } </pre>                                                                                                                                                                                                                                                        |
| 9     | Rover::fence_checks_async() | fence.get_breaches(); fence.check()                                      | Rover IO thread                                              | Function call           | <pre> fence_breaches.last_check_ms = now; const uint8_t orig_breaches = fence.get_breaches(); // check for new breaches; new_breaches is bitmask of fence types breached fence_breaches.new_breaches = fence.check();  if (!fence_breaches.new_breaches &amp;&amp; orig_breaches &amp;&amp; fence.get_breaches() == 0) {     // record clearing of breach     LOGGER_WRITE_ERROR(LogErrorSubsystem::FAILSAFE_FENCE, LogError::ERROR_RESOLVED); } </pre> |

**Layer 2 — Full transitive chain to final consumer (hop count)**

| Ref # | PNT Origin                              | Full Dependency Chain                                                                                                                       | Final Consumer                   | Chain Depth | Notes                                                   |
|-------|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|-------------|---------------------------------------------------------|
| 1     | EKF position health (via AP_AHRS)       | AP_GPS → NavEKF3 → AP_AHRS::have_inertial_nav() → Copter::ekf_has_absolute_position() → Copter::position_ok() → set_mode gate               | flight-mode entry allow/deny     | 5           | Core positioning-health gate.                           |
| 2     | Copter::position_ok()                   | position_ok() → set_mode requires_GPS gate → mode_change_failed() → mode unchanged                                                          | rejected mode change             | 3           | Gate decision path.                                     |
| 3     | AP_AHRS vehicle position (via AC_Fence) | AP::ahrs() position → AC_Fence::check (polygon/circle/alt) → get_breaches bitmask → fence_checks_async latch → Copter::fence_check() action | Copter fence failsafe (RTL/LAND) | 4           | [DUPLICATED] mirrors Rover chain #9.                    |
| 4     | AP_AHRS::get_location()                 | AP_AHRS::get_location() → check_fence_polygon → _poly_loader.breached() → record_breach() → _breached_fences                                | polygon breach bit               | 4           | Polygon position test.                                  |
| 5     | AP_AHRS NE-home position                | AP::ahrs().get_relative_position_NE_home() → check_fence_circle → distance>radius → record_breach()                                         | circle breach bit                | 3           | Circle position test.                                   |
| 6     | AP_AHRS D-home altitude                 | AP::ahrs().get_relative_position_D_home() → check_fence_alt_max → alt>_alt_max → record_breach()                                            | altitude breach bit              | 3           | Altitude position test.                                 |
| 7     | AC_Fence position checks                | position checks → AC_Fence::check() aggregate → breach bitmask → vehicle fence_check() → failsafe action                                    | vehicle fence failsafe           | 4           | Aggregator path.                                        |
| 8     | (no PNT — GPS-independent)              | ModeGuidedNoGPS::init() → angle_control_start() → AC_AttitudeControl → AP_Motors                                                            | AP_Motors                        | 3           | [ABSENT] no positioning hop — fallback by design.       |
| 9     | AP_AHRS vehicle position (via AC_Fence) | AP::ahrs() position → AC_Fence::check → get_breaches → Rover::fence_checks_async latch → Rover::fence_check() action                        | Rover fence failsafe             | 4           | [DUPLICATED] structurally identical to Copter chain #3. |

Table 5 — Indirect / Relational — Navigation (EKF-health watchdog, nav-output reporting &amp; consumption, crash/flight detection)

| # | File Path                | Line(s) | Function / Class             | Trigger Condition                                  | PNT State Observed                                                                              | Behavior Change Description                                                                                                            | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | Notes                                                                                                                                                                                                                                             |
|---|--------------------------|---------|------------------------------|----------------------------------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | ArduCopter/ekf_check.cpp | 117-125 | Copter::ekf_over_threshold() | 10 Hz EKF health check while the EKF has an origin | ahrs.get_variances() → velocity / position / height / magnetometer variances vs g.fs_ekf_thresh | Counts bad iterations; after EKF_CHECK_ITERATIONS_MAX (=10) trips the EKF failsafe (LAND / AltHold).                                   | <pre>float position_var, vel_var, height_var, tas_variance; Vector3f mag_variance; variances_valid = ahrs.get_variances(vel_var, position_var, height_var, mag_variance, tas_variance);  if (!variances_valid) {     return false; }  uint32_t now_us = AP_HAL::micros();</pre>                                                                                                                                                                                                              | INDIRECT/RELATIONAL (navigation). [DUPLICATED] EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk).                    |
| 2 | ArduPlane/ekf_check.cpp  | 56-65   | Plane::ekf_check()           | 10 Hz EKF health check (fixed-wing)                | ekf_over_threshold() (compass & velocity variance vs fs_ekf_thresh)                             | Increments fail_count; two iterations before failsafe asks the EKF to reset yaw (ahrs.request_yaw_reset()); trips EKF failsafe at MAX. | <pre>// compare compass and velocity variance vs threshold if (ekf_over_threshold()) {     // if compass is not yet flagged as bad     if (!ekf_check_state.bad_variance) {         // increase counter         ekf_check_state.fail_count++;         if (ekf_check_state.fail_count == (EKF_CHECK_ITERATION S_MAX-2)) {             // we are two iterations away from declaring a n EK F failsafe, ask the EKF if we can reset // yaw to resolve the issue ahrs.request_yaw_reset();</pre> | INDIRECT/RELATIONAL (navigation). [DUPLICATED] EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk).                    |
| 3 | Rover/ekf_check.cpp      | 93-102  | Rover::ekf_over_threshold()  | 10 Hz EKF health check (ground vehicle)            | ahrs.get_variances() → compass / velocity / position variance; ≥2 over threshold                | Sets bad_variance → Rover EKF failsafe action.                                                                                         | <pre>// use EKF to get variance float position_variance, vel_variance, height_variance, tas_variance; Vector3f mag_variance; ahrs.get_variances(vel_variance, position_variance, height_variance, mag_variance, tas_variance);  // return true if two of compass, velocity and position variances are over the threshold uint8_t over_thresh_count = 0; if (mag_variance.length() &gt;= g.fs_ekf_thresh) {     over_thresh_count++;</pre>                                                    | INDIRECT/RELATIONAL (navigation). [DUPLICATED] EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk).                    |
| 4 | ArduSub/failsafe.cpp     | 112-121 | Sub::failsafe_ekf_check()    | Periodic EKF health check (submarine)              | ahrs.get_variances() → compass_variance & vel_variance vs g.fs_ekf_thresh                       | Clears/sets failsafe.ekf and AP_Notify ekf_bad based on variance vs threshold; GCS warns "EKF bad".                                    | <pre>Vector3f magVar; float compass_variance; float vel_variance; ahrs.get_variances(vel_variance, posVar, hgtVar, magVar, tasVar);  compass_variance = magVar.length();  if (compass_variance &lt; g.fs_ekf_thresh &amp;&amp; vel_variance &lt; g.fs_ekf_thresh) {     last_ekf_good_ms = AP_HAL::millis();     failsafe.ekf = false;     AP_Notify::flags.ekf_bad = false;</pre>                                                                                                           | INDIRECT/RELATIONAL (navigation). [FIX F13 dep-type] [DUPLICATED] EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk). |



| # | File Path                       | Line(s) | Function / Class                                | Trigger Condition                                            | PNT State Observed                                                                             | Behavior Change Description                                                                                                               | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                                                                                                                               | Notes                                                                                                                                                                  |
|---|---------------------------------|---------|-------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 | ArduCopter/crash_check.cpp      | 65-74   | Copter::crash_check() — attitude test           | Disarm/crash monitor running while armed and not landed      | ahrs.cos_roll() / ahrs.cos_pitch() (lean angle); attitude_control->get_att_error_angle_deg()   | Resets crash_counter (declares not-crashing) when lean angle ≤ 15° or attitude error ≤ 30°.                                               | <pre>// check for lean angle over 15 degrees const float lean_angle_deg = degrees(acosf(ahrs.cos_roll()) *ahrs.cos_pitch())); if (lean_angle_deg &lt;= CRASH_CHECK_ANGLE_MIN_DEG) {     crash_counter = 0;     return; }  // check for angle error over 30 degrees const float angle_error = attitude_control-&gt;get_att_error_angle_deg(); if (angle_error &lt;= CRASH_CHECK_ANGLE_DEVIATION_DEG) {</pre>             | [FIX F3] Split row: attitude-only observation (lean L66, angle-error L73). Velocity observation is the separate row #6 (was wrongly bundled into the L64-73 citation). |
| 6 | ArduCopter/crash_check.cpp      | 78-84   | Copter::crash_check() — velocity test           | Continues after the attitude test indicates a possible crash | ahrs.get_velocity_NED(v el_ned) → NED speed magnitude (L80-81)                                 | Resets crash_counter when speed ≥ CRASH_CHECK_SPEED_MAX (10 m/s); otherwise increments toward the 2-second crash trigger → disarm.        | <pre>// check for speed under 10m/s (if available) Vector3f vel_ned; if (ahrs.get_velocity_NED(vel_ned) &amp;&amp; (vel_ned.length() &gt;= CRASH_CHECK_SPEED_MAX)) {     crash_counter = 0;     return; }</pre>                                                                                                                                                                                                         | [FIX F3] Split row: the velocity observation cited at L78-84 (ahrs.get_velocity_NED L80-81), now accurately centered — not the L64-73 attitude range.                  |
| 7 | ArduPlane/GCS_MAVLink_Plane.cpp | 204-212 | GCS_MAVLINK_Plane::send_nav_controller_output() | GCS telemetry stream requests NAV_CONTROLLER_OUTPUT          | nav_controller navigation state (target bearing, xtrack error, wp distance) — unless in MANUAL | Emits the NAV_CONTROLLER_OUTPUT MAVLink message (navigation-health reporting to the GCS).                                                 | <pre>void GCS_MAVLINK_Plane::send_nav_controller_output() const {     if (plane.control_mode == &amp;plane.mode_manual) {         return;     }     #if HAL_QUADPLANE_ENABLED     const QuadPlane &amp;quadplane = plane.quadplane;     if (quadplane.show_vtol_view() &amp;&amp; quadplane.using_wp_nav())     {         const Vector3f &amp;targets = quadplane.attitude_control-&gt;get_att_target_euler_cd();</pre> | INDIRECT/RELATIONAL (navigation). Reporting Sink for navigation state.                                                                                                 |
| 8 | ArduPlane/Attitude.cpp          | 604-611 | Plane::calc_nav_roll()                          | Fixed-wing attitude loop computing the roll demand           | nav_controller->nav_roll_cd() (L1/navigation lateral roll demand)                              | Sets nav_roll_cd from the navigation controller, constrained to ±roll_limit_cd.                                                           | <pre>calculate a new nav_roll_cd from the navigation controller */ void Plane::calc_nav_roll() {     int32_t commanded_roll = nav_controller-&gt;nav_roll_cd();     nav_roll_cd = constrain_int32(commanded_roll, -roll_limit_cd, roll_limit_cd);     update_load_factor(); }</pre>                                                                                                                                     | INDIRECT/RELATIONAL (navigation). Consumes navigation-controller output (user-cited calc_nav_roll).                                                                    |
| 9 | ArduPlane/navigation.cpp        | 86-94   | Plane::navigate()                               | 10 Hz fixed-wing navigation update                           | have_position flag (EKF/GPS position validity); next_WP_loc                                    | Returns early (skips navigation) when have_position is false; otherwise computes the desired direction and distance to the next waypoint. | <pre>void Plane::navigate() {     // do not navigate with corrupt data     // -----     if (!have_position) {         return;     }      if (next_WP_loc.lat == 0 &amp;&amp; next_WP_loc.lng == 0) {</pre>                                                                                                                                                                                                              | INDIRECT/RELATIONAL (navigation). This is the real symbol behind the user-example 'flight_mode_auto' chain (Table 2a L2 #1); gated on PNT position validity.           |

Table 5a — Indirect Navigation dependencies (Layer 1 + Layer 2)

Layer 1 — Direct calls / called-by with typed dependency

| Ref # | Component / Function             | Directly Calls                                                                 | Directly Called By     | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------|----------------------------------|--------------------------------------------------------------------------------|------------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Copter::ekf_over_threshold()     | ahrs.get_variances(); variance filters; compare g.fs_ekf_thresh                | Copter::ekf_check()    | Function call   | <pre> float position_var, vel_var, height_var, tas_variance; Vector3f mag_variance; variances_valid = ahrs.get_variances(vel_var, position_var, height_var, mag_variance, tas_variance);  if (!variances_valid) {     return false; }  uint32_t now_us = AP_HAL::micros(); </pre>                                                                                                                                                                                                                                     |
| 2     | Plane::ekf_check()               | ekf_over_threshold(); ahrs.request_yaw_reset()                                 | Plane 10 Hz scheduler  | Function call   | <pre> // compare compass and velocity variance vs threshold if (ekf_over_threshold()) {     // if compass is not yet flagged as bad     if (!ekf_check_state.bad_variance) {         // increase counter         ekf_check_state.fail_count++;         if (ekf_check_state.fail_count == (EKF_CHECK_ITERATION S_MAX-2)) {             // we are two iterations away from declaring an EK F failsafe, ask the EKF if we can reset             // yaw to resolve the issue             ahrs.request_yaw_reset(); </pre> |
| 3     | Rover::ekf_over_threshold()      | ahrs.get_variances(); compare g.fs_ekf_thresh                                  | Rover::ekf_check()     | Function call   | <pre> // use EKF to get variance float position_variance, vel_variance, height_variance, tas_variance; Vector3f mag_variance; ahrs.get_variances(vel_variance, position_variance, height_variance, mag_variance, tas_variance);  // return true if two of compass, velocity and position variances are over the threshold uint8_t over_thresh_count = 0; if (mag_variance.length() &gt;= g.fs_ekf_thresh) {     over_thresh_count++; </pre>                                                                           |
| 4     | Sub::failsafe_ekf_check()        | ahrs.get_variances(); AP_HAL::millis(); gcs().send_text()                      | Sub periodic scheduler | Function call   | <pre> Vector3f magVar; float compass_variance; float vel_variance; ahrs.get_variances(vel_variance, posVar, hgtVar, magVar, tasVar); compass_variance = magVar.length();  if (compass_variance &lt; g.fs_ekf_thresh &amp;&amp; vel_variance &lt; g.fs_ekf_thresh) {     last_ekf_good_ms = AP_HAL::millis();     failsafe.ekf = false;     AP_Notify::flags.ekf_bad = false; </pre>                                                                                                                                   |
| 5     | Copter::crash_check() (attitude) | ahrs.cos_roll(); ahrs.cos_pitch(); attitude_control->get_att_error_angle_deg() | Copter::crash_check()  | Function call   | <pre> // check for lean angle over 15 degrees const float lean_angle_deg = degrees(acosf(ahrs.cos_roll()*ahrs.cos_pitch())); if (lean_angle_deg &lt;= CRASH_CHECK_ANGLE_MIN_DEG) {     crash_counter = 0;     return; }  // check for angle error over 30 degrees const float angle_error = attitude_control-&gt;get_att_error_angle_deg(); if (angle_error &lt;= CRASH_CHECK_ANGLE_DEVIATION_DEG) { </pre>                                                                                                           |

| Ref # | Component / Function                            | Directly Calls                                             | Directly Called By       | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                                                                                                         |
|-------|-------------------------------------------------|------------------------------------------------------------|--------------------------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6     | Copter::crash_check() (velocity)                | ahrs.get_velocity_NED(vel_ned)                             | Copter::crash_check()    | Function call   | <pre>// check for speed under 10m/s (if available) Vector3f vel_ned; if (ahrs.get_velocity_NED(vel_ned) &amp;&amp; (vel_ned.length() &gt;= CRA SH_CHECK_SPEED_MAX)) {     crash_counter = 0;     return; }</pre>                                                                                                                                                                                                     |
| 7     | GCS_MAVLINK_Plane::send_nav_controller_output() | reads plane.control_mode;<br>nav_controller state          | GCS telemetry scheduler  | Function call   | <pre>void GCS_MAVLINK_Plane::send_nav_controller_output() const {     if (plane.control_mode == &amp;plane.mode_manual) {         return;     }     #if HAL_QUADPLANE_ENABLED     const QuadPlane &amp;quadplane = plane.quadplane;     if (quadplane.show_vtol_view() &amp;&amp; quadplane.using_wp_nav()) {         const Vector3f &amp;targets = quadplane.attitude_control-&gt;get_ att_target_euler_cd();</pre> |
| 8     | Plane::calc_nav_roll()                          | nav_controller->nav_roll_cd();<br>update_load_factor()     | Plane fast attitude loop | Function call   | <pre>calculate a new nav_roll_cd from the navigation controller */ void Plane::calc_nav_roll() {     int32_t commanded_roll = nav_controller-&gt;nav_roll_cd();     nav_roll_cd = constrain_int32(commanded_roll, -roll_limit_cd, roll_limit_cd);     update_load_factor(); }</pre>                                                                                                                                  |
| 9     | Plane::navigate()                               | reads have_position; next_WP_loc;<br>nav_controller update | Plane 10 Hz nav loop     | Data dependency | <pre>void Plane::navigate() {     // do not navigate with corrupt data     // -----     if (!have_position) {         return;     }      if (next_WP_loc.lat == 0 &amp;&amp; next_WP_loc.lng == 0) {</pre>                                                                                                                                                                                                           |

**Layer 2 — Full transitive chain to final consumer (hop count)**

| Ref # | PNT Origin                        | Full Dependency Chain                                                                                                         | Final Consumer               | Chain Depth | Notes                                                                                                                                                                                        |
|-------|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | EKF variances (via AP_AHRS)       | AP_GPS/EKF → ahrs.get_variances() → ekf_over_threshold() → Copter::ekf_check fail_count → failsafe_ekf_event() → LAND/AltHold | EKF failsafe action (LAND)   | 5           | [DUPLICATED] EKF-variance watchdog replicated across ArduCopter/ekf_check.cpp, ArduPlane/ekf_check.cpp, Rover/ekf_check.cpp and ArduSub/failsafe.cpp — a 4-way copy-paste (extraction risk). |
| 2     | EKF variances (Plane)             | EKF variances → Plane::ekf_check → bad_variance → EKF failsafe → mode change (RTL/FBWA)                                       | Plane EKF failsafe           | 4           | [DUPLICATED] mirrors #1.                                                                                                                                                                     |
| 3     | EKF variances (Rover)             | EKF variances → Rover::ekf_over_threshold → Rover::ekf_check → EKF failsafe action                                            | Rover EKF failsafe           | 3           | [DUPLICATED] mirrors #1.                                                                                                                                                                     |
| 4     | EKF variances (Sub)               | EKF variances → Sub::failsafe_ekf_check → failsafe.ekf flag → GCS warn                                                        | Sub EKF failsafe flag        | 3           | [DUPLICATED] mirrors #1.                                                                                                                                                                     |
| 5     | AP_AHRS attitude                  | AHRS attitude → crash_check attitude test → crash_counter → arming.disarm()                                                   | motor disarm                 | 3           | Attitude branch.                                                                                                                                                                             |
| 6     | AP_AHRS NED velocity              | EKF velocity → ahrs.get_velocity_NED() → crash_check speed test → crash_counter → arming.disarm()                             | motor disarm                 | 4           | Velocity branch (F3 split).                                                                                                                                                                  |
| 7     | Navigation controller state       | EKF/nav state → nav_controller → send_nav_controller_output → MAVLink → GCS                                                   | GCS<br>NAV_CONTROLLER_OUTPUT | 4           | Reporting branch.                                                                                                                                                                            |
| 8     | Navigation controller roll demand | nav_controller (L1/EKF) → nav_roll_cd() → calc_nav_roll → nav_roll_cd → attitude controller → SRV_Channels                    | SRV_Channels                 | 5           | Roll-demand consumption.                                                                                                                                                                     |

| Ref # | PNT Origin            | Full Dependency Chain                                                                                           | Final Consumer | Chain Depth | Notes                                                   |
|-------|-----------------------|-----------------------------------------------------------------------------------------------------------------|----------------|-------------|---------------------------------------------------------|
| 9     | EKF position validity | EKF position → have_position → Plane::navigate → nav_controller->update_waypoint → calc_nav_roll → SRV_Channels | SRV_Channels   | 5           | Real symbol behind user-example chain (Table 2a L2 #1). |

Table 6 — Indirect / Relational — Timing (main-loop-lockup watchdogs, PNT-health message hysteresis, rate gating)

| # | File Path               | Line(s) | Function / Class               | Trigger Condition                                                                     | PNT State Observed                                                                           | Behavior Change Description                                                                                                                                  | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                      | Notes                                                                                                                                                                                                                                                                                                                                                                                                             |
|---|-------------------------|---------|--------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | ArduCopter/failsafe.cpp | 37-45   | Copter::failsafe_check()       | 1 kHz timer-failsafe ISR; scheduler tick counter has not advanced since the last call | AP_HAL::micros() (tnow) and AP_Scheduler::ticks() loop counter (timing / loop-cadence state) | When ticks stall and tnow-failsafe_last_timestamp > 2,000,000 us (2 s) sets in_failsafe, drives motors->output_min(), then disarms every 1 s (CPU failsafe). | uint32_t tnow = AP_HAL::micros();<br><br>const uint16_t ticks = scheduler.ticks();<br>if (ticks != failsafe_last_ticks) {<br>// the main loop is running, all is OK<br>failsafe_last_ticks = ticks;<br>failsafe_last_timestamp = tnow;<br>if (in_failsafe) {<br>in_failsafe = false;<br>}<br>} | INDIRECT/RELATIONAL (timing).<br>[DUPLICATED] Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk).                                                                     |
| 2 | ArduPlane/failsafe.cpp  | 22-30   | Plane::failsafe_check()        | 1 kHz timer-failsafe ISR; scheduler tick counter unchanged (main loop stalled)        | micros() (tnow) and AP_Scheduler::ticks() (ticks vs last_ticks)                              | After > 200,000 us (0.2 s) stall sets in_failsafe and passes RC inputs straight to servo outputs (SRV_Channels) every 20 ms instead of disarming.            | uint32_t tnow = micros();<br><br>const uint16_t ticks = scheduler.ticks();<br>if (ticks != last_ticks) {<br>// the main loop is running, all is OK<br>last_ticks = ticks;<br>last_timestamp = tnow;<br>in_failsafe = false;<br>return;<br>}                                                    | INDIRECT/RELATIONAL (timing).<br>[DUPLICATED] Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk).<br>Divergent: 0.2 s trip + RC-passthrough (vs Copter 2 s + disarm). |
| 3 | Rover/failsafe.cpp      | 22-30   | Rover::failsafe_check()        | 1 kHz timer-failsafe ISR; scheduler tick counter unchanged                            | AP_HAL::micros() (tnow) and AP_Scheduler::ticks() (ticks vs last_ticks)                      | After > 200,000 us (0.2 s) stall calls arming.disarm(AP_Arming::Method::CPUFAILSAFE).                                                                        | const uint32_t tnow = AP_HAL::micros();<br><br>const uint16_t ticks = scheduler.ticks();<br>if (ticks != last_ticks) {<br>// the main loop is running, all is OK<br>last_ticks = ticks;<br>last_timestamp = tnow;<br>return;<br>}                                                              | INDIRECT/RELATIONAL (timing).<br>[DUPLICATED] Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk).                                                                     |
| 4 | ArduSub/failsafe.cpp    | 31-39   | Sub::mainloop_failsafe_check() | 1 kHz timer-failsafe ISR; scheduler tick counter unchanged                            | AP_HAL::micros() (tnow) and AP_Scheduler::ticks() loop counter                               | After > 2,000,000 us (2 s) stall sets in_failsafe and drives motors.output_min().                                                                            | uint32_t tnow = AP_HAL::micros();<br><br>const uint16_t ticks = scheduler.ticks();<br>if (ticks != failsafe_last_ticks) {<br>// the main loop is running, all is OK<br>failsafe_last_ticks = ticks;<br>failsafe_last_timestamp = tnow;<br>if (in_failsafe) {<br>in_failsafe = false;<br>}<br>} | INDIRECT/RELATIONAL (timing).<br>[DUPLICATED] Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk).                                                                     |

| # | File Path                | Line(s) | Function / Class                                   | Trigger Condition                                                            | PNT State Observed                                                                             | Behavior Change Description                                                                                                                                                                                       | Code Snippet (5-10 lines)                                                                                                                                                                                                                                                                                           | Notes                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---|--------------------------|---------|----------------------------------------------------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5 | Blimp/failsafe.cpp       | 37-45   | Blimp::failsafe_check()                            | 1 kHz timer-failsafe ISR; scheduler tick counter unchanged                   | AP_HAL::micros() (tnow) and AP_Scheduler::ticks() loop counter                                 | After > 2,000,000 us (2 s) stall sets in_failsafe and drives motors->output_min().                                                                                                                                | <pre>uint32_t tnow = AP_HAL::micros();  const uint16_t ticks = scheduler.ticks(); if (ticks != failsafe_last_ticks) {     // the main loop is running, all is OK     failsafe_last_ticks = ticks;     failsafe_last_timestamp = tnow;     if (in_failsafe) {         in_failsafe = false;     } }</pre>             | INDIRECT/RELATIONAL (timing). [DUPLICATED] Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk). Blimp included only because it observes scheduler timing state (AAP 0.3.2). |
| 6 | ArduCopter/ekf_check.cpp | 84-88   | Copter::ekf_check() — GCS warn throttle            | EKF variance already flagged bad AND ≥ 30 s elapsed since the last warning   | AP_HAL::millis() - ekf_check_state.last_warn_time vs EKF_CHECK_WARNING_TIME (30 s)             | Rate-limits the “EKF variance” GCS_SEVERITY_CRITICAL text to at most once per 30 s; stamps last_warn_time = millis().                                                                                             | <pre>// send message to gcs if ((AP_HAL::millis() - ekf_check_state.last_warn_time) &gt; EKF_CHECK_WARNING_TIME) {     gcs().send_text(MAV_SEVERITY_CRITICAL, "EKF variance");     ekf_check_state.last_warn_time = AP_HAL::millis(); }</pre>                                                                       | INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 5 #1 ekf_check]. [DUPLICATED] 30 s warn-throttle copied in ArduPlane/ekf_check.cpp.                                                                                                                                                                                                                                                                             |
| 7 | ArduPlane/ekf_check.cpp  | 78-82   | Plane::ekf_check() — GCS warn throttle             | EKF variance already flagged bad AND ≥ 30 s elapsed since the last warning   | AP_HAL::millis() - ekf_check_state.last_warn_time vs EKF_CHECK_WARNING_TIME (30 s)             | Rate-limits the “EKF variance” GCS text to at most once per 30 s; stamps last_warn_time = millis().                                                                                                               | <pre>// send message to gcs if ((AP_HAL::millis() - ekf_check_state.last_warn_time) &gt; EKF_CHECK_WARNING_TIME) {     gcs().send_text(MAV_SEVERITY_CRITICAL, "EKF variance");     ekf_check_state.last_warn_time = AP_HAL::millis(); }</pre>                                                                       | INDIRECT/RELATIONAL (timing). [DUPLICATED] mirrors ArduCopter/ekf_check.cpp:84-88.                                                                                                                                                                                                                                                                                                                                     |
| 8 | ArduSub/failsafe.cpp     | 125-130 | Sub::failsafe_ekf_check() — 2 s confirm hysteresis | EKF variance bad but not yet bad for a full 2 s                              | AP_HAL::millis() vs last_ekf_good_ms + 2000 (2 s clock-based confirmation window)              | Suppresses the EKF failsafe (clears failsafe.ekf / ekf_bad and returns) until the EKF has been bad for 2 solid seconds, then trips → arming.disarm(EKFFAILSAFE).                                                  | <pre>// Bad EKF for 2 solid seconds triggers failsafe if (AP_HAL::millis() &lt; last_ekf_good_ms + 2000) {     failsafe.ekf = false;     AP_Notify::flags.ekf_bad = false;     return; }</pre>                                                                                                                      | INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 5 #4]. Divergent hysteresis: Sub uses a clock-based 2 s window + 20 s GCS warn-throttle (L141), whereas Copter/Plane use a count-based EKF_CHECK_ITERATIONS_MAX (≈1 s) + 30 s warn-throttle.                                                                                                                                                                    |
| 9 | ArduCopter/ekf_check.cpp | 125-132 | Copter::ekf_over_threshold() — dt filtering        | Each ekf_over_threshold() evaluation (nominally the 10 Hz ekf_check cadence) | AP_HAL::micros() delta → dt = (now_us - last_ekf_check_us) * 1e-6 (actual inter-call interval) | Applies the measured dt to the position/velocity variance low-pass filters (pos_variance_filt / vel_variance_filt); the effective filter cutoff tracks the real scheduler call interval rather than a fixed rate. | <pre>uint32_t now_us = AP_HAL::micros(); float dt = (now_us - last_ekf_check_us) * 1e-6f;  // always update filtered values as this serves the vibration check as well position_var = pos_variance_filt.apply(position_var, dt); vel_var = vel_variance_filt.apply(vel_var, dt);  last_ekf_check_us = now_us;</pre> | INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 5 #1]. EKF check-rate ↔ scheduler coupling (AAP 0.5.5): filter behaviour depends on scheduler timing.                                                                                                                                                                                                                                                           |

| #  | File Path            | Line(s) | Function / Class             | Trigger Condition             | PNT State Observed                                                                                                | Behavior Change Description                                                                                                                    | Code Snippet (5-10 lines)                                                                                                                                                                                                                                 | Notes                                                                                                                                                   |
|----|----------------------|---------|------------------------------|-------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 10 | ArduCopter/fence.cpp | 8-16    | Copter::fence_checks_async() | 1 kHz async fence IO callback | AP_HAL::millis() (now) and AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 40U) (40 ms / 25 Hz gate)   | Returns early until 40 ms have elapsed, throttling the geofence breach check to 25 Hz; the position-vs-fence observation itself is Table 4 #3. | <pre>void Copter::fence_checks_async() {     const uint32_t now = AP_HAL::millis();      if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now , 40U)) { // 25Hz update rate         return;     }      if (fence_breaches.have_updates) {</pre> | INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 4 #3] — timing side of the same function (F1). [DUPLICATED] vs Rover/fence.cpp (divergent rate). |
| 11 | Rover/fence.cpp      | 6-14    | Rover::fence_checks_async()  | 1 kHz async fence IO callback | AP_HAL::millis() (now) and AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 100U) (100 ms / 10 Hz gate) | Returns early until 100 ms have elapsed, throttling the geofence breach check to 10 Hz.                                                        | <pre>void Rover::fence_checks_async() {     const uint32_t now = AP_HAL::millis();      if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now , 100U)) { // 10Hz update rate         return;     }      if (fence_breaches.have_updates) {</pre> | INDIRECT/RELATIONAL (timing). [MULTI-CATEGORY w/ Table 4 #9]. [DUPLICATED] vs ArduCopter/fence.cpp:8-16 — divergent rate (40U/25 Hz vs 100U/10 Hz).     |

Table 6a — Indirect Timing dependencies (Layer 1 + Layer 2)

## Layer 1 — Direct calls / called-by with typed dependency

| Ref # | Component / Function           | Directly Calls                                                                               | Directly Called By                                                                           | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                    |
|-------|--------------------------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Copter::failsafe_check()       | AP_HAL::micros();<br>AP_Scheduler::ticks();<br>motors->output_min();<br>motors->armed(false) | failsafe_check_static() <-<br>hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:87)  | Function call   | <pre>uint32_t tnow = AP_HAL::micros();  const uint16_t ticks = scheduler.ticks(); if (ticks != failsafe_last_ticks) {     // the main loop is running, all is OK     failsafe_last_ticks = ticks;     failsafe_last_timestamp = tnow;     if (in_failsafe) {         in_failsafe = false;</pre> |
| 2     | Plane::failsafe_check()        | micros(); AP_Scheduler::ticks();<br>rc().read_input();<br>SRV_Channels::set_output_scaled()  | failsafe_check_static() <-<br>hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:103) | Function call   | <pre>uint32_t tnow = micros();  const uint16_t ticks = scheduler.ticks(); if (ticks != last_ticks) {     // the main loop is running, all is OK     last_ticks = ticks;     last_timestamp = tnow;     in_failsafe = false;     return;</pre>                                                   |
| 3     | Rover::failsafe_check()        | AP_HAL::micros();<br>AP_Scheduler::ticks();<br>arming.disarm(CPUFAILSAFE)                    | failsafe_check_static() <-<br>hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:111) | Function call   | <pre>const uint32_t tnow = AP_HAL::micros();  const uint16_t ticks = scheduler.ticks(); if (ticks != last_ticks) {     // the main loop is running, all is OK     last_ticks = ticks;     last_timestamp = tnow;     return; }</pre>                                                            |
| 4     | Sub::mainloop_failsafe_check() | AP_HAL::micros();<br>AP_Scheduler::ticks();<br>motors.output_min()                           | failsafe_check_static() <-<br>hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:71)  | Function call   | <pre>uint32_t tnow = AP_HAL::micros();  const uint16_t ticks = scheduler.ticks(); if (ticks != failsafe_last_ticks) {     // the main loop is running, all is OK     failsafe_last_ticks = ticks;     failsafe_last_timestamp = tnow;     if (in_failsafe) {         in_failsafe = false;</pre> |

| Ref # | Component / Function                       | Directly Calls                                                               | Directly Called By                                                                          | Dependency Type | Code Snippet                                                                                                                                                                                                                                                                                                                  |
|-------|--------------------------------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5     | Blimp::failsafe_check()                    | AP_HAL::micros();<br>AP_Scheduler::ticks();<br>motors->output_min()          | failsafe_check_static() <-<br>hal.scheduler->register_timer_failsafe (1 kHz, system.cpp:59) | Function call   | uint32_t tnow = AP_HAL::micros();<br><br>const uint16_t ticks = scheduler.ticks();<br>if (ticks != failsafe_last_ticks) {<br>// the main loop is running, all is OK<br>failsafe_last_ticks = ticks;<br>failsafe_last_timestamp = tnow;<br>if (in_failsafe) {<br>in_failsafe = false;<br>}<br>}                                |
| 6     | Copter::ekf_check() (warn throttle)        | AP_HAL::millis(); gcs().send_text()                                          | Copter::ekf_check() bad-variance branch                                                     | Function call   | // send message to gcs<br>if ((AP_HAL::millis() - ekf_check_state.last_warn_time) > EKF_CHECK_WARNING_TIME) {<br>gcs().send_text(MAV_SEVERITY_CRITICAL, "EKF variance");<br>ekf_check_state.last_warn_time = AP_HAL::millis();<br>}                                                                                           |
| 7     | Plane::ekf_check() (warn throttle)         | AP_HAL::millis(); gcs().send_text()                                          | Plane::ekf_check() bad-variance branch                                                      | Function call   | // send message to gcs<br>if ((AP_HAL::millis() - ekf_check_state.last_warn_time) > EKF_CHECK_WARNING_TIME) {<br>gcs().send_text(MAV_SEVERITY_CRITICAL, "EKF variance");<br>ekf_check_state.last_warn_time = AP_HAL::millis();<br>}                                                                                           |
| 8     | Sub::failsafe_ekf_check() (2 s hysteresis) | AP_HAL::millis(); compares last_ekf_good_ms + 2000                           | Sub periodic scheduler                                                                      | Function call   | // Bad EKF for 2 solid seconds triggers failsafe<br>if (AP_HAL::millis() < last_ekf_good_ms + 2000) {<br>failsafe.ekf = false;<br>AP_Notify::flags.ekf_bad = false;<br>return;<br>}                                                                                                                                           |
| 9     | Copter::ekf_over_threshold() (dt)          | AP_HAL::micros();<br>pos_variance_filt.apply();<br>vel_variance_filt.apply() | Copter::ekf_check()                                                                         | Function call   | uint32_t now_us = AP_HAL::micros();<br>float dt = (now_us - last_ekf_check_us) * 1e-6f;<br><br>// always update filtered values as this serves the vibration check as well<br>position_var = pos_variance_filt.apply(position_var, dt);<br>vel_var = vel_variance_filt.apply(vel_var, dt);<br><br>last_ekf_check_us = now_us; |
| 10    | Copter::fence_checks_async()               | AP_HAL::millis();<br>AP_HAL::timeout_expired(...,40U)                        | 1 kHz async IO process (hal.scheduler register_io_process)                                  | Function call   | void Copter::fence_checks_async()<br>{<br>const uint32_t now = AP_HAL::millis();<br><br>if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 40U)) { // 25Hz update rate<br>return;<br>}<br><br>if (fence_breaches.have_updates) {                                                                                 |
| 11    | Rover::fence_checks_async()                | AP_HAL::millis();<br>AP_HAL::timeout_expired(...,100U)                       | 1 kHz async IO process (hal.scheduler register_io_process)                                  | Function call   | void Rover::fence_checks_async()<br>{<br>const uint32_t now = AP_HAL::millis();<br><br>if (!AP_HAL::timeout_expired(fence_breaches.last_check_ms, now, 100U)) { // 10Hz update rate<br>return;<br>}<br><br>if (fence_breaches.have_updates) {                                                                                 |

**Layer 2 — Full transitive chain to final consumer (hop count)**



| Ref # | PNT Origin                                   | Full Dependency Chain                                                                                                                                                    | Final Consumer                        | Chain Depth | Notes                                                                                                                                                                                                                                                                                                        |
|-------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | AP_HAL::micros() + AP_Scheduler tick counter | AP_Scheduler::ticks() / AP_HAL::micros() → failsafe_check_static() [1 kHz timer-failsafe] → Copter::failsafe_check() (>2 s stall) → AP_Motors::output_min()/armed(false) | AP_Motors (motor min / disarm)        | 3           | [DUPLICATED] Main-loop-lockup watchdog replicated 5 ways — ArduCopter/failsafe.cpp, ArduPlane/failsafe.cpp, Rover/failsafe.cpp, ArduSub/failsafe.cpp (mainloop_failsafe_check) and Blimp/failsafe.cpp — each reads AP_Scheduler::ticks() + AP_HAL::micros(); divergent thresholds/actions (extraction risk). |
| 2     | AP_HAL::micros() + AP_Scheduler tick counter | AP_Scheduler::ticks() / micros() → failsafe_check_static() → Plane::failsafe_check() (>0.2 s stall) → SRV_Channels::set_output_scaled() → servos_output()                | SRV_Channels (RC passthrough)         | 4           | [DUPLICATED] watchdog; divergent 0.2 s trip + RC-passthrough action.                                                                                                                                                                                                                                         |
| 3     | AP_HAL::micros() + AP_Scheduler tick counter | AP_Scheduler::ticks() / micros() → failsafe_check_static() → Rover::failsafe_check() (>0.2 s stall) → AP_Arming::disarm(CPUFAILSAFE)                                     | AP_Arming (disarm)                    | 3           | [DUPLICATED] watchdog.                                                                                                                                                                                                                                                                                       |
| 4     | AP_HAL::micros() + AP_Scheduler tick counter | AP_Scheduler::ticks() / micros() → failsafe_check_static() → Sub::mainloop_failsafe_check() (>2 s stall) → AP_Motors::output_min()                                       | AP_Motors (motor min)                 | 3           | [DUPLICATED] watchdog.                                                                                                                                                                                                                                                                                       |
| 5     | AP_HAL::micros() + AP_Scheduler tick counter | AP_Scheduler::ticks() / micros() → failsafe_check_static() → Blimp::failsafe_check() (>2 s stall) → AP_Motors::output_min()                                              | AP_Motors (motor min)                 | 3           | [DUPLICATED] watchdog; Blimp observes scheduler timing state only (AAP 0.3.2).                                                                                                                                                                                                                               |
| 6     | AP_HAL::millis() wall clock                  | AP_HAL::millis() → Copter::ekf_check() (>30 s since last_warn) → gcs().send_text("EKF variance") → GCS_MAVLINK                                                           | GCS_MAVLINK (telemetry text)          | 3           | [DUPLICATED] 30 s warn-throttle (Copter/Plane). [MULTI-CATEGORY w/ Table 5 #1].                                                                                                                                                                                                                              |
| 7     | AP_HAL::millis() wall clock                  | AP_HAL::millis() → Plane::ekf_check() (>30 s since last_warn) → gcs().send_text("EKF variance") → GCS_MAVLINK                                                            | GCS_MAVLINK (telemetry text)          | 3           | [DUPLICATED] mirrors #6.                                                                                                                                                                                                                                                                                     |
| 8     | AP_HAL::millis() wall clock                  | AP_HAL::millis() → Sub::failsafe_ekf_check() (EKF bad ≥ 2 s) → failsafe.ekf = true → arming.disarm(EKFFAILSAFE)                                                          | AP_Arming (EKF failsafe disarm)       | 3           | [MULTI-CATEGORY w/ Table 5 #4]. 2 s confirm + 20 s warn-throttle.                                                                                                                                                                                                                                            |
| 9     | AP_HAL::micros() inter-call interval         | AP_HAL::micros() → dt → pos/vel_variance_filt.apply(var, dt) → ekf_over_threshold() → Copter::ekf_check() failsafe decision                                              | Copter::ekf_check() variance decision | 4           | EKF check-rate <-> scheduler coupling (AAP 0.5.5). [MULTI-CATEGORY w/ Table 5 #1].                                                                                                                                                                                                                           |
| 10    | AP_HAL::millis() wall clock                  | AP_HAL::millis() → AP_HAL::timeout_expired(40U) gate → Copter::fence_checks_async() → AC_Fence::check() → fence_breaches latched → main-loop fence failsafe              | AC_Fence / fence failsafe             | 5           | [MULTI-CATEGORY w/ Table 4 #3]. [DUPLICATED] vs Rover (40U/25 Hz).                                                                                                                                                                                                                                           |
| 11    | AP_HAL::millis() wall clock                  | AP_HAL::millis() → AP_HAL::timeout_expired(100U) gate → Rover::fence_checks_async() → AC_Fence::check() → fence_breaches latched                                         | AC_Fence / fence failsafe             | 4           | [MULTI-CATEGORY w/ Table 4 #9]. [DUPLICATED] vs Copter (100U/10 Hz).                                                                                                                                                                                                                                         |

## COVERAGE & EXPLICIT-ABSENCE REGISTER

Exhaustive survey ledger: every required surface is either catalogued above (with table/row references) or recorded here as a checked explicit absence. Non-inference discipline (no “handled elsewhere”).

### Indirect supporting-library register (AAP 0.3.1) — AC\_Fence / AP\_BattMonitor / AP\_RCProtocol: non-PNT triggers whose failsafe action is PNT-gated, plus checked absences.

| Surface / File                                                                   | PNT Intersection | Detail / Predicate / Behavior                                                                                                                                                                                      | Evidence (path:line · cross-ref)                                                                                  |
|----------------------------------------------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| libraries/AC_Fence/ (AC_Fence::check + check_fence_polygon / check_fence_circle) | ✓ Catalogued     | Reads vehicle position vs polygon / circle / altitude fence; position-dependent breach. Fully catalogued in the numbered tables (not an absence).                                                                  | libraries/AC_Fence/AC_Fence.cpp:679-843 · Table 4 #4-#8 / Table 4a                                                |
| libraries/AP_BattMonitor/ (58 source files; AP_BattMonitor::check_failsafes)     | [ABSENT] checked | Directory-wide token sweep found ZERO PNT reads (no ahrs. / gps. / get_location / position_ok / get_velocity_NED). Failsafe trigger is voltage / capacity only; PNT enters at the vehicle action layer, not here.  | libraries/AP_BattMonitor/AP_BattMonitor.cpp:781 (check_failsafes) · action cross-ref ArduCopter/events.cpp:99-121 |
| libraries/AP_RCProtocol/ (RC failsafe flag)                                      | [ABSENT] checked | Token sweep found ZERO PNT reads. Failsafe is a boolean signal-present flag (failsafe_active); no position / nav / PNT-timing observation. PNT enters at the vehicle action layer.                                 | libraries/AP_RCProtocol/AP_RCProtocol.h:126-130 (failsafe_active) · action cross-ref ArduCopter/events.cpp:99-121 |
| ArduCopter/events.cpp (handle_battery_failsafe / radio / GCS FS action select)   | ✓ PNT-reactive   | Non-PNT trigger (battery / RC / GCS) but the chosen failsafe action (RTL / SmartRTL) is gated on position_ok and degrades to LAND when position is invalid — the relational PNT site for batt / RC / GCS failsafe. | ArduCopter/events.cpp:99-121 (do_failsafe_action BATTERY_FAILSAFE) · gate Table 4 #2                              |

### Cross-vehicle indirect safety-surface register (F16) — Rover / ArduSub failsafe, fence, and mode surfaces, each resolved to its catalogued numbered-table row.

| Surface / File                                                | PNT Intersection | Detail / Predicate / Behavior                                                                                                    | Evidence (path:line · cross-ref)                                                |
|---------------------------------------------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Rover/failsafe.cpp (Rover::failsafe_check main-loop watchdog) | ✓ Catalogued     | Main-loop-lockup watchdog reading scheduler ticks + micros; 0.2 s trip → arming.disarm(CPUFAILSAFE). Catalogued, not an absence. | Rover/failsafe.cpp:18-31 · Table 6 #3 / Table 6a #3                             |
| Rover/fence.cpp (Rover::fence_checks_async)                   | ✓ Catalogued     | Geofence position-vs-boundary check on a 100 ms (10 Hz) timing gate; position-dependent breach. Catalogued.                      | Rover/fence.cpp:6-14 · Table 6 #11 / Table 6a #11                               |
| ArduSub/failsafe.cpp (Sub::failsafe_ekf_check)                | ✓ Catalogued     | EKF-variance health watchdog with 2 s confirm + 20 s warn hysteresis. Catalogued.                                                | ArduSub/failsafe.cpp:100-150 · Table 5 #4 / Table 6 #8                          |
| ArduSub/failsafe.cpp (Sub::mainloop_failsafe_check)           | ✓ Catalogued     | Main-loop-lockup watchdog (scheduler ticks + micros; 2 s → motors.output_min). Catalogued.                                       | ArduSub/failsafe.cpp:29-55 · Table 6 #4 / Table 6a #4                           |
| ArduSub/mode*.cpp (12 files)                                  | ✓ Catalogued     | All 12 Sub mode files enumerated with per-file requires_GPS gating and checked absences in the Sub mode-gating matrix below.     | ArduSub/mode.cpp:89-90 (set_mode gate) · Sub mode-gating matrix (this appendix) |

### AntennaTracker & Blimp PNT-intersection register (F18; AAP 0.3.2 — included only where they observe / consume PNT): presence sites, per-mode gating, and checked absences.

| Surface / File                                                      | PNT Intersection | Detail / Predicate / Behavior                                                                                                                                            | Evidence (path:line · cross-ref)                                                       |
|---------------------------------------------------------------------|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| AntennaTracker/tracking.cpp (Tracker::update_vehicle_pos_estimate)  | ✓ PNT consumer   | Reads vehicle.location and vehicle.vel and dead-reckons location_estimate (offset by vel times dt) to point the antenna. Direct PNT consumption (presence, not absence). | AntennaTracker/tracking.cpp:7-26                                                       |
| AntennaTracker/mode_auto.cpp + mode_guided.cpp                      | ✓ PNT consumer   | Auto / Guided pointing modes consume the tracked vehicle position / bearing to drive pitch and yaw. PNT-reactive.                                                        | AntennaTracker/mode_auto.cpp:5-10 · consumes tracking.cpp:7-26                         |
| AntennaTracker/mode_manual.cpp + mode_scan.cpp + mode_servotest.cpp | [ABSENT] checked | Manual / Scan / Servotest drive the antenna by stick / pattern / test only; token sweep found ZERO vehicle-position reads. Checked absence.                              | AntennaTracker/mode_servotest.cpp (0 PNT tokens) · mode_scan.cpp / mode_manual.cpp (0) |
| Blimp/mode.cpp (Blimp::set_mode position gate)                      | ✓ Gate enforcer  | Rejects PNT modes when position invalid: requires_GPS() and not blimp.position_ok(). Same gate idiom as Copter / Sub.                                                    | Blimp/mode.cpp:78-79                                                                   |
| Blimp/mode_velocity.cpp (ModeVelocity)                              | ✓ Position-gated | requires_GPS()==true → needs a valid position / velocity estimate to enter.                                                                                              | Blimp/mode.h:181 (requires_GPS=true) · gate Blimp/mode.cpp:78-79                       |
| Blimp/mode_loiter.cpp (ModeLoiter)                                  | ✓ Position-gated | requires_GPS()==true → position-gated.                                                                                                                                   | Blimp/mode.h:226 (requires_GPS=true) · gate Blimp/mode.cpp:78-79                       |

| Surface / File                                             | PNT Intersection | Detail / Predicate / Behavior                                                                                                | Evidence (path:line · cross-ref)                                    |
|------------------------------------------------------------|------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Blimp/mode_rtl.cpp (ModeRTL)</b>                        | ✓ Position-gated | requires_GPS()=true → position-gated.                                                                                        | Blimp/mode.h:315 (requires_GPS=true) · gate<br>Blimp/mode.cpp:78-79 |
| <b>Blimp/mode_manual.cpp (ModeManual)</b>                  | [ABSENT] checked | requires_GPS()=false → no position gate; manual stick control. Checked absence.                                              | Blimp/mode.h:138 (requires_GPS=false)                               |
| <b>Blimp/mode_land.cpp (ModeLand)</b>                      | [ABSENT] checked | requires_GPS()=false → no horizontal-position gate (descent uses non-PNT references).<br>Checked absence for horizontal PNT. | Blimp/mode.h:271 (requires_GPS=false)                               |
| <b>Blimp/failsafe.cpp (Blimp::failsafe_check_watchdog)</b> | ✓ Catalogued     | Main-loop-lockup watchdog (scheduler ticks + micros; 2 s → output_min). Catalogued.                                          | Blimp/failsafe.cpp:37-45 · Table 6 #5 / Table 6a #5                 |

### Copter flight modes PNT-gating matrix — 29 files (21 PNT-relevant, 8 checked-absence). Predicate: requires\_GPS().

| Surface / File                                     | PNT Intersection          | Detail / Predicate / Behavior                                                                                                    | Evidence (path:line · cross-ref)                                                                                     |
|----------------------------------------------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>mode.cpp (Copter::set_mode() position gate)</b> | ✓ Gate enforcer           | new_flightmode->requires_GPS() && !copter.position_ok() rejects PNT modes when position invalid                                  | ArduCopter/mode.cpp:303-307 · Table 4 #2                                                                             |
| <b>mode_acro.cpp (ModeAcro)</b>                    | [ABSENT] checked          | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).          | ArduCopter/mode.h:428 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)                                             |
| <b>mode_acro_heli.cpp (ModeAcro_Heli)</b>          | [ABSENT] checked          | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).          | ArduCopter/mode.h:428 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)                                             |
| <b>mode_althold.cpp (ModeAltHold)</b>              | ✓ PNT (no horiz-GPS gate) | requires_GPS()=false, but consumes PNT here (pos_control ×9; e.g. altitude pos_control / optical-flow / optional position).      | ArduCopter/mode.h:484 · use<br>ArduCopter/mode_althold.cpp:13 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)     |
| <b>mode_auto.cpp (ModeAuto)</b>                    | ✓ Position-gated          | requires_GPS()=true (conditional: all sub-modes except NAV_ATTITUDE_TIME) — PNT position/nav required to enter; behaviour gated. | ArduCopter/mode_auto.cpp:178 · use<br>ArduCopter/mode_auto.cpp:42 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307) |
| <b>mode_autorotate.cpp (ModeAutorotate)</b>        | [ABSENT] checked          | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).          | ArduCopter/mode.h:2047 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)                                            |
| <b>mode_autotune.cpp (ModeAutoTune)</b>            | ✓ PNT (no horiz-GPS gate) | requires_GPS()=false, but consumes PNT here (pos_control ×5; e.g. altitude pos_control / optical-flow / optional position).      | ArduCopter/mode.h:829 · use<br>ArduCopter/mode_autotune.cpp:31 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)    |
| <b>mode_avoid_adsb.cpp (ModeAvoidADSB)</b>         | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                       | ArduCopter/mode.h:1888 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)                                            |
| <b>mode_brake.cpp (ModeBrake)</b>                  | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                       | ArduCopter/mode.h:854 · use<br>ArduCopter/mode_brake.cpp:13 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)       |
| <b>mode_circle.cpp (ModeCircle)</b>                | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                       | ArduCopter/mode.h:884 · use<br>ArduCopter/mode_circle.cpp:16 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)      |
| <b>mode_drift.cpp (ModeDrift)</b>                  | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                       | ArduCopter/mode.h:914 · use ArduCopter/mode_drift.cpp:54<br>· gate Table 4 #2 (ArduCopter/mode.cpp:303-307)          |
| <b>mode_flip.cpp (ModeFlip)</b>                    | [ABSENT] checked          | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).          | ArduCopter/mode.h:942 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)                                             |
| <b>mode_flowhold.cpp (ModeFlowHold)</b>            | ✓ PNT (no horiz-GPS gate) | requires_GPS()=false, but consumes PNT here (pos_control ×12; e.g. altitude pos_control / optical-flow / optional position).     | ArduCopter/mode.h:989 · use<br>ArduCopter/mode_flowhold.cpp:92 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)    |
| <b>mode_follow.cpp (ModeFollow)</b>                | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                       | ArduCopter/mode.h:1918 · use<br>ArduCopter/mode_follow.cpp:33 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)     |
| <b>mode_guided.cpp (ModeGuided)</b>                | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                       | ArduCopter/mode.h:1077 · use<br>ArduCopter/mode_guided.cpp:77 · gate Table 4 #2<br>(ArduCopter/mode.cpp:303-307)     |

| Surface / File                                      | PNT Intersection          | Detail / Predicate / Behavior                                                                                                | Evidence (path:line · cross-ref)                                                                              |
|-----------------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>mode_guided_custom.cpp (ModeGuidedCustom)</b>    | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                   | ArduCopter/mode.h:1077 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)                                        |
| <b>mode_guided_nogps.cpp (ModeGuidedNoGPS)</b>      | [ABSENT] checked          | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).      | ArduCopter/mode.h:1253 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)                                        |
| <b>mode_land.cpp (ModeLand)</b>                     | ✓ PNT (no horiz-GPS gate) | requires_GPS()=false, but consumes PNT here (position_ok ×9; e.g. altitude pos_control / optical-flow / optional position).  | ArduCopter/mode.h:1277 · use ArduCopter/mode_land.cpp:7 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)       |
| <b>mode_loiter.cpp (ModeLoiter)</b>                 | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                   | ArduCopter/mode.h:1323 · use ArduCopter/mode_loiter.cpp:17 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)    |
| <b>mode_poshold.cpp (ModePosHold)</b>               | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                   | ArduCopter/mode.h:1373 · use ArduCopter/mode_poshold.cpp:28 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)   |
| <b>mode_rtl.cpp (ModeRTL)</b>                       | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                   | ArduCopter/mode.h:1461 · use ArduCopter/mode_rtl.cpp:21 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)       |
| <b>mode_smart_rtl.cpp (ModeSmartRTL)</b>            | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                   | ArduCopter/mode.h:1577 · use ArduCopter/mode_smart_rtl.cpp:16 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307) |
| <b>mode_sport.cpp (ModeSport)</b>                   | ✓ PNT (no horiz-GPS gate) | requires_GPS()=false, but consumes PNT here (pos_control ×9; e.g. altitude pos_control / optical-flow / optional position).  | ArduCopter/mode.h:1637 · use ArduCopter/mode_sport.cpp:13 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)     |
| <b>mode_stabilize.cpp (ModeStabilize)</b>           | [ABSENT] checked          | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).      | ArduCopter/mode.h:1664 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)                                        |
| <b>mode_stabilize_heli.cpp (ModeStabilize_Heli)</b> | [ABSENT] checked          | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).      | ArduCopter/mode.h:1664 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)                                        |
| <b>mode_systemid.cpp (ModeSystemId)</b>             | ✓ PNT (no horiz-GPS gate) | requires_GPS()=false, but consumes PNT here (pos_control ×22; e.g. altitude pos_control / optical-flow / optional position). | ArduCopter/mode.h:1712 · use ArduCopter/mode_systemid.cpp:115 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307) |
| <b>mode_throw.cpp (ModeThrow)</b>                   | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                   | ArduCopter/mode.h:1791 · use ArduCopter/mode_throw.cpp:23 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)     |
| <b>mode_turtle.cpp (ModeTurtle)</b>                 | [ABSENT] checked          | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).      | ArduCopter/mode.h:1849 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)                                        |
| <b>mode_zigzag.cpp (ModeZigZag)</b>                 | ✓ Position-gated          | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                                   | ArduCopter/mode.h:1967 · use ArduCopter/mode_zigzag.cpp:75 · gate Table 4 #2 (ArduCopter/mode.cpp:303-307)    |

### Plane flight modes PNT-gating matrix — 26 files (20 PNT-relevant, 6 checked-absence). Predicate: `does_auto_navigation()`.

| Surface / File                                      | PNT Intersection | Detail / Predicate / Behavior                                                                                                   | Evidence (path:line · cross-ref)                                                             |
|-----------------------------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>mode.cpp (Plane::set_mode() / mode enter)</b>    | ✓ Gate enforcer  | does_auto_navigation() consulted; nav modes consume EKF/AHRS nav state                                                          | ArduPlane/mode.cpp:144                                                                       |
| <b>mode_LoiterAltQLand.cpp (ModeLoiterAltQLand)</b> | ✓ Position-gated | does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.                                              | ArduPlane/mode.h:427 · use ArduPlane/mode_LoiterAltQLand.cpp:8 · gate ArduPlane/mode.cpp:144 |
| <b>mode_acro.cpp (ModeAcro)</b>                     | [ABSENT] checked | does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence). | gate ArduPlane/mode.cpp:144                                                                  |
| <b>mode_auto.cpp (ModeAuto)</b>                     | ✓ Position-gated | does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.                                              | ArduPlane/mode_auto.cpp:137 · use ArduPlane/mode_auto.cpp:10 · gate ArduPlane/mode.cpp:144   |

| Surface / File                              | PNT Intersection          | Detail / Predicate / Behavior                                                                                                         | Evidence (path:line · cross-ref)                                                              |
|---------------------------------------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>mode_autoland.cpp (ModeAutoLand)</b>     | ✓ Position-gated          | does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.                                                    | ArduPlane/mode.h:957 · use<br>ArduPlane/mode_autoland.cpp:79 · gate<br>ArduPlane/mode.cpp:144 |
| <b>mode_autotune.cpp (ModeAutoTune)</b>     | [ABSENT] checked          | does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).       | gate ArduPlane/mode.cpp:144                                                                   |
| <b>mode_avoidADSB.cpp (ModeAvoidADSB)</b>   | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (update_loiter ×1; e.g. altitude pos_control / optical-flow / optional position). | use ArduPlane/mode_avoidADSB.cpp:19 · gate<br>ArduPlane/mode.cpp:144                          |
| <b>mode_circle.cpp (ModeCircle)</b>         | ✓ Position-gated          | does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.                                                    | ArduPlane/mode.h:399 · use ArduPlane/mode_circle.cpp:7 · gate<br>ArduPlane/mode.cpp:144       |
| <b>mode_cruise.cpp (ModeCruise)</b>         | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (prev_WP_loc ×4; e.g. altitude pos_control / optical-flow / optional position).   | use ArduPlane/mode_cruise.cpp:83 · gate<br>ArduPlane/mode.cpp:144                             |
| <b>mode_fbwa.cpp (ModeFBWA)</b>             | [ABSENT] checked          | does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).       | gate ArduPlane/mode.cpp:144                                                                   |
| <b>mode_fbwb.cpp (ModeFBWB)</b>             | [ABSENT] checked          | does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).       | gate ArduPlane/mode.cpp:144                                                                   |
| <b>mode_guided.cpp (ModeGuided)</b>         | ✓ Position-gated          | does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.                                                    | ArduPlane/mode.h:360 · use<br>ArduPlane/mode_guided.cpp:11 · gate<br>ArduPlane/mode.cpp:144   |
| <b>mode_loiter.cpp (ModeLoiter)</b>         | ✓ Position-gated          | does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.                                                    | ArduPlane/mode.h:427 · use ArduPlane/mode_loiter.cpp:7 · gate<br>ArduPlane/mode.cpp:144       |
| <b>mode_manual.cpp (ModeManual)</b>         | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (quadplane. ×1; e.g. altitude pos_control / optical-flow / optional position).    | use ArduPlane/mode_manual.cpp:26 · gate<br>ArduPlane/mode.cpp:144                             |
| <b>mode_qacro.cpp (ModeQAcro)</b>           | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (quadplane. ×16; e.g. altitude pos_control / optical-flow / optional position).   | use ArduPlane/mode_qacro.cpp:8 · gate<br>ArduPlane/mode.cpp:144                               |
| <b>mode_qautotune.cpp (ModeQAutotune)</b>   | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (quadplane. ×4; e.g. altitude pos_control / optical-flow / optional position).    | use ArduPlane/mode_qautotune.cpp:11 · gate<br>ArduPlane/mode.cpp:144                          |
| <b>mode_qhover.cpp (ModeQHover)</b>         | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (pos_control ×13; e.g. altitude pos_control / optical-flow / optional position).  | use ArduPlane/mode_qhover.cpp:9 · gate<br>ArduPlane/mode.cpp:144                              |
| <b>mode_qland.cpp (ModeQLand)</b>           | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (quadplane. ×5; e.g. altitude pos_control / optical-flow / optional position).    | use ArduPlane/mode_qland.cpp:9 · gate<br>ArduPlane/mode.cpp:144                               |
| <b>mode_qloiter.cpp (ModeQLoiter)</b>       | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (loiter_nav ×52; e.g. altitude pos_control / optical-flow / optional position).   | use ArduPlane/mode_qloiter.cpp:9 · gate<br>ArduPlane/mode.cpp:144                             |
| <b>mode_qrtl.cpp (ModeQRTL)</b>             | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (quadplane. ×47; e.g. altitude pos_control / optical-flow / optional position).   | use ArduPlane/mode_qrtl.cpp:10 · gate<br>ArduPlane/mode.cpp:144                               |
| <b>mode_qstabilize.cpp (ModeQStabilize)</b> | ✓ PNT (no horiz-GPS gate) | does_auto_navigation()=false, but consumes PNT here (quadplane. ×19; e.g. altitude pos_control / optical-flow / optional position).   | use ArduPlane/mode_qstabilize.cpp:8 · gate<br>ArduPlane/mode.cpp:144                          |
| <b>mode_rtl.cpp (ModeRTL)</b>               | ✓ Position-gated          | does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.                                                    | ArduPlane/mode.h:507 · use ArduPlane/mode_rtl.cpp:6 · gate<br>ArduPlane/mode.cpp:144          |
| <b>mode_stabilize.cpp (ModeStabilize)</b>   | [ABSENT] checked          | does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).       | gate ArduPlane/mode.cpp:144                                                                   |
| <b>mode_takeoff.cpp (ModeTakeoff)</b>       | ✓ Position-gated          | does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.                                                    | ArduPlane/mode.h:909 · use<br>ArduPlane/mode_takeoff.cpp:74 · gate<br>ArduPlane/mode.cpp:144  |
| <b>mode_thermal.cpp (ModeThermal)</b>       | ✓ Position-gated          | does_auto_navigation()=true — PNT position/nav required to enter; behaviour gated.                                                    | ArduPlane/mode.h:1025 · use<br>ArduPlane/mode_thermal.cpp:16 · gate<br>ArduPlane/mode.cpp:144 |

| Surface / File                          | PNT Intersection | Detail / Predicate / Behavior                                                                                                   | Evidence (path:line · cross-ref) |
|-----------------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| <b>mode_training.cpp (ModeTraining)</b> | [ABSENT] checked | does_auto_navigation()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence). | gate ArduPlane/mode.cpp:144      |

### Rover drive modes PNT-gating matrix — 14 files (11 PNT-relevant, 3 checked-absence). Predicate: requires\_position().

| Surface / File                                | PNT Intersection          | Detail / Predicate / Behavior                                                                                                | Evidence (path:line · cross-ref)                                          |
|-----------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <b>mode.cpp (Mode::enter() position gate)</b> | ✓ Gate enforcer           | rover.ekf_position_ok() && requires_position()/requires_velocity() gate entry                                                | Rover/mode.cpp:32-38                                                      |
| <b>mode_acro.cpp (ModeAcro)</b>               | ✓ PNT (no horiz-GPS gate) | requires_position()=false, but consumes PNT here (wp_nav ×1; e.g. altitude pos_control / optical-flow / optional position).  | Rover/mode.h:235 · use Rover/mode_acro.cpp:37 · gate Rover/mode.cpp:32-38 |
| <b>mode_auto.cpp (ModeAuto)</b>               | ✓ Position-gated          | requires_position()=true — PNT position/nav required to enter; behaviour gated.                                              | use Rover/mode_auto.cpp:14 · gate Rover/mode.cpp:32-38                    |
| <b>mode_circle.cpp (ModeCircle)</b>           | ✓ Position-gated          | requires_position()=true — PNT position/nav required to enter; behaviour gated.                                              | use Rover/mode_circle.cpp:85 · gate Rover/mode.cpp:32-38                  |
| <b>mode_dock.cpp (ModeDock)</b>               | ✓ Position-gated          | requires_position()=true — PNT position/nav required to enter; behaviour gated.                                              | use Rover/mode_dock.cpp:75 · gate Rover/mode.cpp:32-38                    |
| <b>mode_follow.cpp (ModeFollow)</b>           | ✓ Position-gated          | requires_position()=true — PNT position/nav required to enter; behaviour gated.                                              | use Rover/mode_follow.cpp:12 · gate Rover/mode.cpp:32-38                  |
| <b>mode_guided.cpp (ModeGuided)</b>           | ✓ Position-gated          | requires_position()=true — PNT position/nav required to enter; behaviour gated.                                              | use Rover/mode_guided.cpp:15 · gate Rover/mode.cpp:32-38                  |
| <b>mode_hold.cpp (ModeHold)</b>               | [ABSENT] checked          | requires_position()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence). | Rover/mode.h:630 · gate Rover/mode.cpp:32-38                              |
| <b>mode_loiter.cpp (ModeLoiter)</b>           | ✓ Position-gated          | requires_position()=true — PNT position/nav required to enter; behaviour gated.                                              | use Rover/mode_loiter.cpp:6 · gate Rover/mode.cpp:32-38                   |
| <b>mode_manual.cpp (ModeManual)</b>           | [ABSENT] checked          | requires_position()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence). | Rover/mode.h:681 · gate Rover/mode.cpp:32-38                              |
| <b>mode_rtl.cpp (ModeRTL)</b>                 | ✓ Position-gated          | requires_position()=true — PNT position/nav required to enter; behaviour gated.                                              | use Rover/mode_rtl.cpp:11 · gate Rover/mode.cpp:32-38                     |
| <b>mode_simple.cpp (ModeSimple)</b>           | ✓ Position-gated          | requires_position()=true — PNT position/nav required to enter; behaviour gated.                                              | gate Rover/mode.cpp:32-38                                                 |
| <b>mode_smart_rtl.cpp (ModeSmartRTL)</b>      | ✓ Position-gated          | requires_position()=true — PNT position/nav required to enter; behaviour gated.                                              | use Rover/mode_smart_rtl.cpp:17 · gate Rover/mode.cpp:32-38               |
| <b>mode_steering.cpp (ModeSteering)</b>       | [ABSENT] checked          | requires_position()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence). | Rover/mode.h:785 · gate Rover/mode.cpp:32-38                              |

### Sub flight modes PNT-gating matrix — 12 files (7 PNT-relevant, 5 checked-absence). Predicate: requires\_GPS().

| Surface / File                                  | PNT Intersection | Detail / Predicate / Behavior                                                                                           | Evidence (path:line · cross-ref)                                                  |
|-------------------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>mode.cpp (Sub::set_mode() position gate)</b> | ✓ Gate enforcer  | new_flightmode->requires_GPS() && !sub.position_ok() rejects PNT modes                                                  | ArduSub/mode.cpp:89-90                                                            |
| <b>mode_acro.cpp (ModeAcro)</b>                 | [ABSENT] checked | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence). | ArduSub/mode.h:155 · gate ArduSub/mode.cpp:89-90                                  |
| <b>mode_althold.cpp (ModeAlthold)</b>           | [ABSENT] checked | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence). | ArduSub/mode.h:201 · gate ArduSub/mode.cpp:89-90                                  |
| <b>mode_auto.cpp (ModeAuto)</b>                 | ✓ Position-gated | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                              | ArduSub/mode.h:314 · use ArduSub/mode_auto.cpp:11 · gate ArduSub/mode.cpp:89-90   |
| <b>mode_circle.cpp (ModeCircle)</b>             | ✓ Position-gated | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                              | ArduSub/mode.h:382 · use ArduSub/mode_circle.cpp:10 · gate ArduSub/mode.cpp:89-90 |
| <b>mode_guided.cpp (ModeGuided)</b>             | ✓ Position-gated | requires_GPS()=true — PNT position/nav required to enter; behaviour gated.                                              | ArduSub/mode.h:260 · use ArduSub/mode_guided.cpp:34 · gate ArduSub/mode.cpp:89-90 |
| <b>mode_manual.cpp (ModeManual)</b>             | [ABSENT] checked | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence). | ArduSub/mode.h:132 · gate ArduSub/mode.cpp:89-90                                  |
| <b>mode_motordetect.cpp (ModeMotordetect)</b>   | [ABSENT] checked | requires_GPS()=false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence). | ArduSub/mode.h:427 · gate ArduSub/mode.cpp:89-90                                  |



| Surface / File                            | PNT Intersection          | Detail / Predicate / Behavior                                                                                                | Evidence (path:line · cross-ref)                                                    |
|-------------------------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>mode_poshold.cpp (ModePoshold)</b>     | ✓ Position-gated          | requires_GPS()==true — PNT position/nav required to enter; behaviour gated.                                                  | ArduSub/mode.h:355 · use ArduSub/mode_poshold.cpp:13 · gate ArduSub/mode.cpp:89-90  |
| <b>mode_stabilize.cpp (ModeStabilize)</b> | [ABSENT] checked          | requires_GPS()==false and zero position/nav/velocity tokens — manual/attitude mode; no PNT dependence (checked absence).     | ArduSub/mode.h:178 · gate ArduSub/mode.cpp:89-90                                    |
| <b>mode_surface.cpp (ModeSurface)</b>     | ✓ PNT (no horiz-GPS gate) | requires_GPS()==false, but consumes PNT here (wp_nav ×1; e.g. altitude pos_control / optical-flow / optional position).      | ArduSub/mode.h:404 · use ArduSub/mode_surface.cpp:54 · gate ArduSub/mode.cpp:89-90  |
| <b>mode_surftrak.cpp (ModeSurftrak)</b>   | ✓ PNT (no horiz-GPS gate) | requires_GPS()==false, but consumes PNT here (pos_control ×3; e.g. altitude pos_control / optical-flow / optional position). | ArduSub/mode.h:201 · use ArduSub/mode_surftrak.cpp:91 · gate ArduSub/mode.cpp:89-90 |